MIT | Academy of Engineering
(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

DATABASE MANAGEMENT SYSTEM

Teaching Notes

Lecture Number	Topic to be covered
1	**Unit 5 - Transaction Processing& Hashing (08 Hrs)** ➢ Transaction Concept , A simple Transaction Model
2	➢ Transaction Atomicity and Durability, Transaction Isolation
3	➢ ACID Properties, ➢ Serializability
4	➢ Concurrency Control Techniques: Lock based Protocols
5	➢ Deadlock handling, Multiple Granularity
6	➢ Time stamp-Based Protocols
7	➢ Recovery System

DEEPTI BHIRUD, MIT AoE, ALANDI

Database systems

Unit 5 – Transaction Processing

1. Transaction Concept:

Transactions are a set of operations that are used to perform some logical set of work. A transaction is made to change data in a database which can be done by inserting new data, updating the existing data, or by deleting the data that is no longer required.

Example: Suppose an employee of bank transfers Rs 800 from X's account to Y's account. This small transaction contains several low-level tasks:

X's Account

1. Open_Account(X)
2. Old_Balance = X.balance
3. New_Balance = Old_Balance - 800
4. X.balance = New_Balance
5. Close_Account(X)

Y's Account

1. Open_Account(Y)
2. Old_Balance = Y.balance
3. New_Balance = Old_Balance + 800
4. Y.balance = New_Balance
5. Close_Account(Y)
6.

Operations of Transaction:

Following are the main operations of transaction:

Read(X): Read operation is used to read the value of X from the database and stores it in a buffer in main memory.

Write(X): Write operation is used to write the value back to the database from the buffer.

Example :

To debit transaction from an account which consists of following operations:

1. R(X);
2. X = X - 500;
3. W(X);

Let's assume the value of X before starting of the transaction is 4000.

- The first operation reads X's value from database and stores it in a buffer.
- The second operation will decrease the value of X by 500. So buffer will contain 3500.
- The third operation will write the buffer's value to the database. So X's final value will be 3500.

But it may be possible that because of the failure of hardware, software or power, etc. that transaction may fail before finished all the operations in the set.

For example:

If in the above transaction, the debit transaction fails after executing operation 2 then X's value will remain 4000 in the database which is not acceptable by the bank.

To solve this problem, we have two important operations:

Commit: It is used to save the work done permanently.

Rollback: It is used to undo the work done.

Two main issues to deal in transaction:

1. Failures of various kinds, such as hardware failures and system crashes
2. Concurrent execution of multiple transactions

2. Simple transaction model

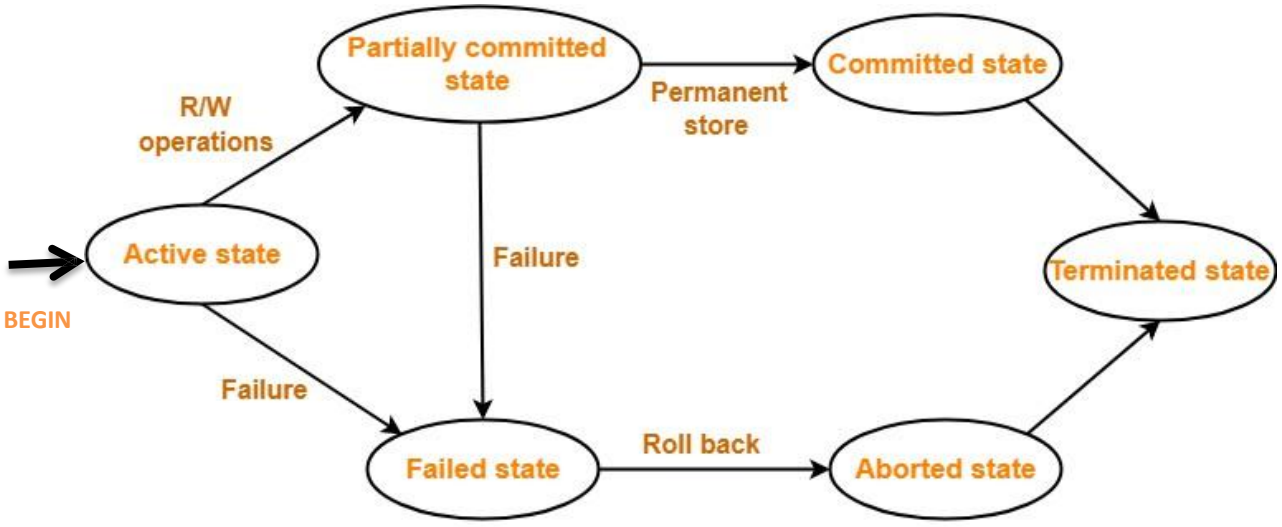
Simple transaction model is a model of transaction of how it must be. It has active, partially committed, failed, aborted, and committed states. Transaction is a several operations that can change the content of the database which is handled by a single program. Simple transaction model follows all ACID properties while doing transactions.

Transaction states

A transaction goes through many different states throughout its life cycle.

These states are called as **transaction states**.

Transaction states are as follows-



Transaction States in DBMS

Active state

- The active state is the first state of every transaction. In this state, the transaction is being executed.
- A transaction is called in an active state as long as its instructions are getting executed.

- All the changes made by the transaction now are stored in the buffer in main memory.
- For example: Insertion or deletion or updating a record is done here. But all the records are still not saved to the database.

Partially committed

- After the last instruction of transaction has executed, it enters into a **partially committed state**.
- After entering this state, the transaction is considered to be partially committed.
- It is not considered fully committed because all the changes made by the transaction are still stored in the buffer in main memory.
- In the partially committed state, a transaction executes its final operation, but the data is still not saved to the database.
- In the total mark calculation example, a final display of the total marks step is executed in this state.

Committed

- After all the changes made by the transaction have been successfully stored into the database, it enters into a **committed state**.
- Now, the transaction is considered to be fully committed.
- In this state, all the effects are now permanently saved on the database system.

Failed state

- When a transaction is getting executed in the active state or partially committed state and some failure occurs due to which it becomes impossible to continue the execution, it enters into a **failed state**.
- In the example of total mark calculation, if the database is not able to fire a query to fetch the marks, then the transaction will fail to execute.

Aborted

- After the transaction has failed and entered into a failed state, all the changes made by it have to be undone.
- To undo the changes made by the transaction, it becomes necessary to roll back the transaction to bring the database into a consistent state.
- After the transaction has rolled back completely, it enters into an **aborted state**.
- If the transaction fails in the middle of the transaction then before executing the transaction, all the executed transactions are rolled back to its consistent state.
- After aborting the transaction, the database recovery module will select one of the two operations:
    1. Re-start the transaction
    2. Kill the transaction

Terminated State-

This is the last state in the life cycle of a transaction.

- After entering the committed state or aborted state, the transaction finally enters into a **terminated state** where its life cycle finally comes to an end.

NOTE-

- After a transaction has entered the committed state, it is not possible to roll back the transaction.
- In other words, it is not possible to undo the changes that has been made by the transaction.
- This is because the system is updated into a new consistent state.

- The only way to undo the changes is by carrying out another transaction called as **compensating transaction** that performs the reverse operations.

3. ACID Properties-

A transaction in a database system must maintain Atomicity, Consistency, Isolation, and Durability − commonly known as ACID properties − in order to ensure accuracy, completeness, and data integrity.

Atomicity-

- This property ensures that a transaction must be treated as an atomic unit, that is, either the transaction occurs completely or it does not occur at all.
- It states that all operations of the transaction take place at once if not, the transaction is aborted.
- There is no midway, i.e., the transaction cannot occur partially. That is why, it is also referred to as **"All or nothing rule".**
- Each transaction is treated as one unit and either run to completion or is not executed at all.
- It is the responsibility of Transaction Control Manager to ensure atomicity of the transactions.

Atomicity involves the following two operations:
Abort: If a transaction aborts then all the changes made are not visible.
Commit: If a transaction commits then all the changes made are visible.

Example: Let's assume that following transaction T consisting of T1 and T2. A consists of Rs 600 and B consists of Rs 300. Transfer Rs 100 from account A to account B.

T1	**T2**
Read(A)	Read(B)
A:= A-100	B:= B+100
Write(A)	Write(B)

After completion of the transaction, A consists of Rs 500 and B consists of Rs 400.
If the transaction T fails after the completion of transaction T1 but before completion of transaction T2, then the amount will be deducted from A but not added to B. This shows the inconsistent database state. In order to ensure correctness of database state, the transaction must be executed in entirety.

Consistency

- The integrity constraints are maintained so that the database is consistent before and after the transaction.
- The execution of a transaction will leave a database in either its prior stable state or a new stable state.
- The consistent property of database states that every transaction sees a consistent database instance.
- The transaction is used to transform the database from one consistent state to another consistent state.

For example: The total amount must be maintained before or after the transaction.

1. Total before T occurs = 600+300=900
2. Total after T occurs= 500+400=900

Therefore, the database is consistent. In the case when T1 is completed but T2 fails, then inconsistency will occur.

Isolation

- In a database system where more than one transaction are being executed simultaneously and in parallel, the property of isolation states that all the transactions will be carried out and executed as if it is the only transaction in the system.
- No transaction will affect the existence of any other transaction.
- It shows that the data which is used at the time of execution of a transaction cannot be used by the second transaction until the first one is completed.
- In isolation, if the transaction T1 is being executed and using the data item X, then that data item can't be accessed by any other transaction T2 until the transaction T1 ends.
- The concurrency control subsystem of the DBMS enforced the isolation property.

Durability

- The database should be durable enough to hold all its latest updates even if the system fails or restarts.
- If a transaction updates a chunk of data in a database and commits, then the database will hold the modified data.
- If a transaction commits but the system fails before the data could be written on to the disk, then that data will be updated once the system springs back into action.
- The durability property is used to indicate the performance of the database's consistent state. It states that the transaction made the permanent changes.
- They cannot be lost by the erroneous operation of a faulty transaction or by the system failure. When a transaction is completed, then the database reaches a state known as the consistent state.
- That consistent state cannot be lost, even in the event of a system's failure.
- The recovery subsystem of the DBMS has the responsibility of Durability property.

4. Implementation of atomicity and durability

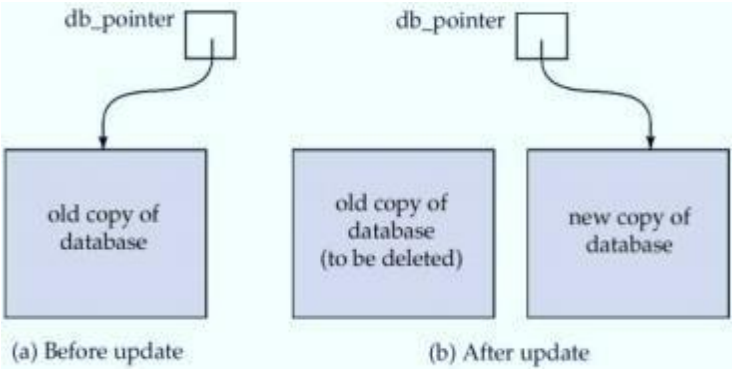
The recovery-management component of a database system can support atomicity and durability by a variety of schemes.One scheme is using shadow copy

Shadow copy:

- In the shadow-copy scheme, a transaction that wants to update the database first creates a complete copy of the database. All updates are done on the new database copy, leaving the original copy, untouched. If at any point the transaction has to be aborted, the system merely deletes the new copy. The old copy of the database has not been affected.
- This scheme is based on making copies of the database, called shadow copies, assumes that only one transaction is active at a time. The scheme also assumes that the database is simply a file on disk. A pointer called db-pointer is maintained on disk; it points to the current copy of the database.

If the transaction completes, it is committed as follows:

- First, the operating system is asked to make sure that all pages of the new copy of the database have been written out to disk.
- After the operating system has written all the pages to disk, the database system updates the pointer db-pointer to point to the new copy of the database; the new copy then becomes the current copy of the database. The old copy of the database is then deleted.
- The transaction is said to have been committed at the point where the updated db pointer is written to disk.



How the technique handles transaction failures:

- If the transaction fails at any time before db-pointer is updated, the old contents of the database are not affected.
- We can abort the transaction by just deleting the new copy of the database.
- Once the transaction has been committed, all the updates that it performed are in the database pointed to by db pointer.
- Thus, either all updates of the transaction are reflected, or none of the effects are reflected, regardless of transaction failure.

How the technique handles system failures:

- Suppose that the system fails at any time before the updated db-pointer is written to disk. Then, when the system restarts, it will read db-pointer and will thus see the original contents of the database, and none of the effects of the transaction will be visible on the database.
- Next, suppose that the system fails after db-pointer has been updated on disk. Before the pointer is updated, all updated pages of the new copy of the database were written to disk.
- Again, we assume that, once a file is written to disk, its contents will not be damaged even if there is a system failure. Therefore, when the system restarts, it will read db-pointer and will thus see the contents of the database after all the updates performed by the transaction.

5. Transaction Isolation

Isolation determines how transaction integrity is visible to other users and systems. It means that a transaction should take place in a system in such a way that it is the only transaction that is accessing the resources in a database system.
Isolation levels define the degree to which a transaction must be isolated from the data modifications made by any other transaction in the database system. A transaction isolation level is defined by the following phenomena –

- **Dirty Read –** A Dirty read is a situation when a transaction reads data that has not yet been committed. For example, Let's say transaction 1 updates a row and leaves it uncommitted, meanwhile, Transaction 2 reads the updated row. If transaction 1 rolls back the change, transaction 2 will have read data that is considered never to have existed.
- **Non Repeatable read –** Non Repeatable read occurs when a transaction reads the same row twice and gets a different value each time. For example, suppose transaction T1 reads data. Due to concurrency, another transaction T2 updates the same data and commit, Now if transaction T1 rereads the same data, it will retrieve a different value.
- **Phantom Read –** Phantom Read occurs when two same queries are executed, but the rows retrieved by the two, are different. For example, suppose transaction T1 retrieves a set of rows that satisfy some search criteria. Now, Transaction T2 generates some

new rows that match the search criteria for transaction T1. If transaction T1 re-executes the statement that reads the rows, it gets a different set of rows this time.

Based on these phenomena, The SQL standard defines four isolation levels :

1. **Read Uncommitted –** Read Uncommitted is the lowest isolation level. In this level, one transaction may read not yet committed changes made by other transactions, thereby allowing dirty reads. At this level, transactions are not isolated from each other.
2. **Read Committed –** This isolation level guarantees that any data read is committed at the moment it is read. Thus it does not allow dirty read. The transaction holds a read or write lock on the current row, and thus prevents other transactions from reading, updating, or deleting it.
3. **Repeatable Read –** This is the most restrictive isolation level. The transaction holds read locks on all rows it references and writes locks on referenced rows for update and delete actions. Since other transactions cannot read, update or delete these rows, consequently it avoids non-repeatable read.
4. **Serializable –** This is the highest isolation level. A *serializable* execution is guaranteed to be serializable. Serializable execution is defined to be an execution of operations in which concurrently executing transactions appears to be serially executing.

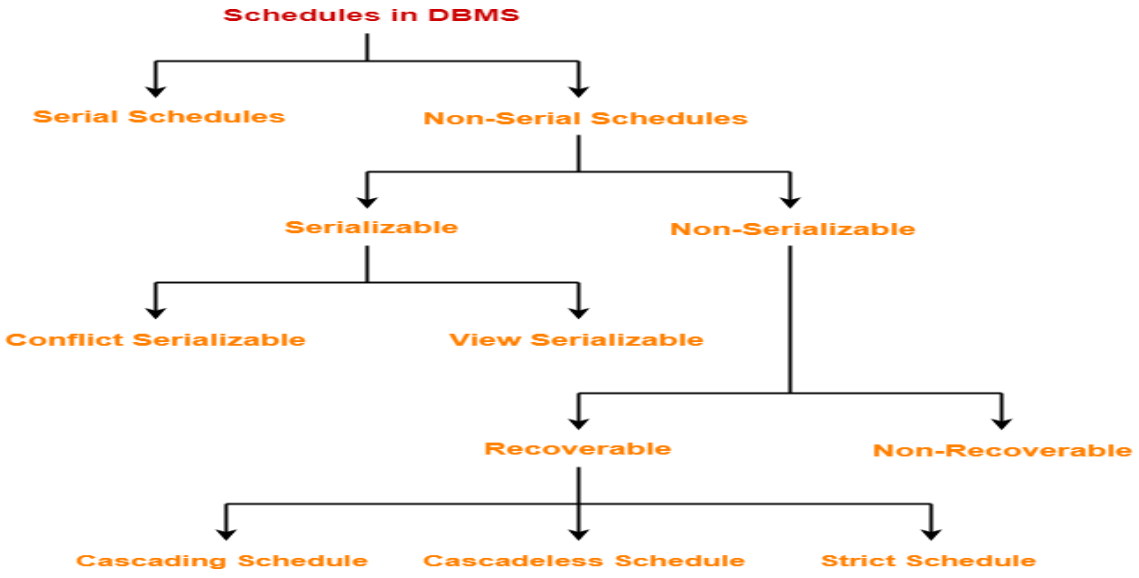
Schedule

- A schedule is the order in which the operations of multiple transactions appear for execution.
  - Serial schedules are always consistent.
  - Non-serial schedules are not always consistent.

Transactions are a set of instructions that perform operations on databases. When multiple transactions are running concurrently, then a sequence is needed in which the operations are to be performed because at a time, only one operation can be performed on the database. This sequence of operations is known as Schedule, and this process is known as Scheduling.

Types of Schedules-

In DBMS, schedules may be classified as-



1. Serial Schedule

All the transactions are executed serially one after the other. In serial Schedule, a transaction does not start execution until the currently running transaction finishes execution. This type of execution of the transaction is also known as non-interleaved execution. Serial Schedule are always recoverable, cascades, strict and consistent. A serial schedule always gives the correct result.

Transaction T1	Transaction T2
R(A)	
W(A)	
R(B)	
W(B)	
commit	
	R(A)
	W(B)
	commit

For example: Consider two transactions T1 and T2 shown above, which perform some operations. If it has no interleaving of operations, then there are the following two possible outcomes –

- Execute all T1 operations, which were followed by all T2 operations.
- Execute all T2 operations, which were followed by all T1 operations.

In the above figure, the Schedule shows the serial Schedule where T1 is followed by T2, i.e. T1 -> T2. Where R(A) -> reading some data item 'A'. And, W(B) -> writing/updating some data item 'B'.

If n = number of transactions, then a number of serial schedules possible = n!.
Therefore, for the above 2 transactions, a total number of serial schedules possible = 2.

Non-serial Schedule

In a non-serial Schedule, multiple transactions execute concurrently/simultaneously, unlike the serial Schedule, where one transaction must wait for another to complete all its operations. In the Non-Serial Schedule, the other transaction proceeds without the completion of the previous transaction. All the transaction operations are interleaved or mixed with each other.

Non-serial schedules are NOT always recoverable, cascades, strict and consistent.

Transaction T1	Transaction T2
R1(A)	
W1(A)	
	R2(A)
	W2(A)
R1(B)	
W1(B)	
	R2(B)
	W2(B)

S1 (Schedule 1)

In this Schedule, there are two transactions, T1 and T2, executing concurrently. The operations of T1 and T2 are interleaved. So, this Schedule is an example of a Non-Serial Schedule.

Total number of non-serial schedules = Total number of schedules – Total number of serial schedules

Non-serial schedules are further categorized into serializable and non-serializable schedules.

6. Serializability

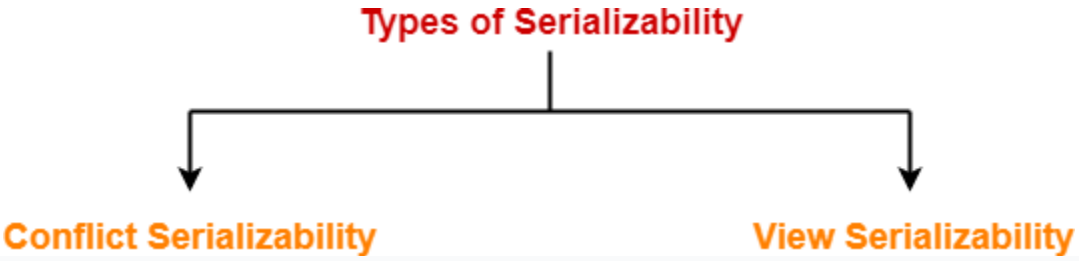
Serializability is a concept that helps to identify which non-serial schedules are correct and will maintain the consistency of the database. A serializable schedule always leaves the database in a consistent state. A serial schedule is always a serializable schedule because, in a serial Schedule, a transaction only starts when the other transaction has finished execution. ***A non-serial schedule of n transactions is said to be a serializable schedule, if it is equivalent to the serial Schedule of those n transactions.*** A serial schedule does not allow concurrency; only one transaction executes at a time, and the other starts when the already running transaction is finished.

Characteristics-

Serial schedules are always-

- Consistent
- Recoverable
- Cascadeless
- Strict

Types of Serializability:



Conflict Serializability

A schedule is called conflict serializable if it can be transformed into a serial schedule by swapping non-conflicting operations.

Conflicting Operations-

Two operations are called as **conflicting operations** if all the following conditions hold true for them-

- Both the operations belong to different transactions
- Both the operations are on the same data item
- At least one of the two operations is a write operation

Transaction T1	Transaction T2
Read(A)	
Write(A)	
	Read(A)
	Write(A)
Read(B)	
Write(B)	
	Read(B)
	Write(B)

Before Swapping

Transaction T1	Transaction T2
Read(A)	
Write(A)	
Read(B)	
Write(B)	
	Read(A)
	Write(A)
	Read(B)
	Write(B)

After Swapping

In this schedule, Write(A)/Read(A) and Write(B)/Read(B) are called as conflicting operations. This is because all the above conditions hold true for them. Thus, by swapping(non-conflicting) 2nd pair of the read/write operation of 'A' data item and 1st pair of the read/write operation of 'B' data item, this non-serial Schedule can be converted into a serializable Schedule. Therefore, *it is* conflict serializable.

Consider the following schedule-

Transaction T1	Transaction T2
R1 (A)	
W1 (A)	
	R2 (A)
R1 (B)	

In this schedule,

- W1 (A) and R2 (A) are called as conflicting operations.
- This is because all the above conditions hold true for them.

Checking Whether a Schedule is Conflict Serializable Or Not-

Follow the following steps to check whether a given non-serial schedule is conflict serializable or not-

Step-01:
Find and list all the conflicting operations.
Step-02:
Start creating a precedence graph by drawing one node for each transaction.
Step-03:
Draw an edge for each conflict pair such that if X_i (V) and Y_j (V) forms a conflict pair then draw an edge from T_i to T_j.This ensures that T_i gets executed before T_j.
Step-04:
Check if there is any cycle formed in the graph.
If there is no cycle found, then the schedule is conflict serializable otherwise not.

Problem-01: Check whether the given schedule S is conflict serializable or not-

S : R1(A) , R2(A) , R1(B) , R2(B) , R3(B) , W1(A) , W2(B)

T1	T2	T3
R1(A)		
	R2(A)	
R1(B)		
	R2(B)	
		R3(B)
W1(A)		
	W2(B)	

Solution-

Step-01:

List all the conflicting operations and determine the dependency between the transactions-

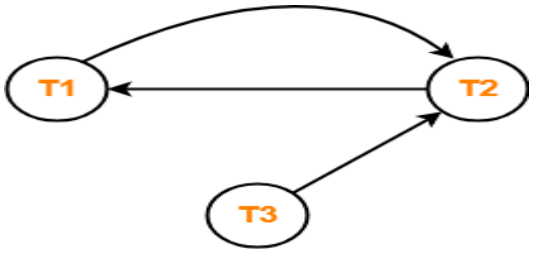
R2(A) , W1(A) (T2 → T1)

R1(B) , W2(B) (T1 → T2)

R3(B) , W2(B) (T3 → T2)

Step-02:

Draw the precedence graph-

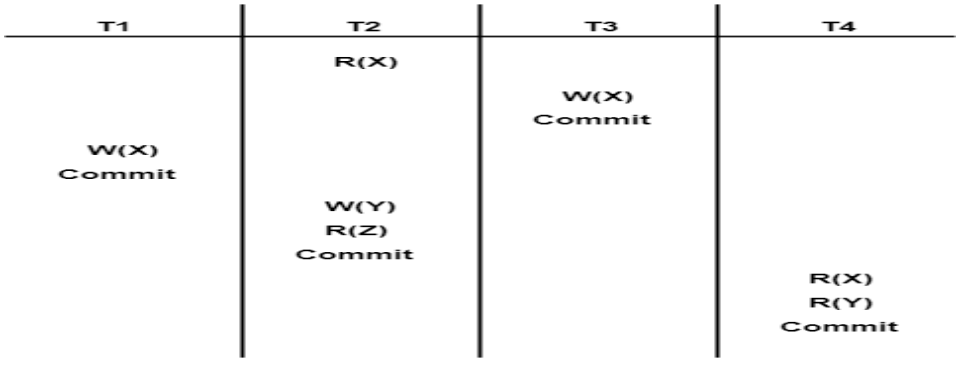


Clearly, there exists a cycle in the precedence graph.

Therefore, the given schedule S is not conflict serializable.

Problem-02:

Check whether the given schedule S is conflict serializable and recoverable or not-



Solution-

Checking Whether S is Conflict Serializable Or Not-

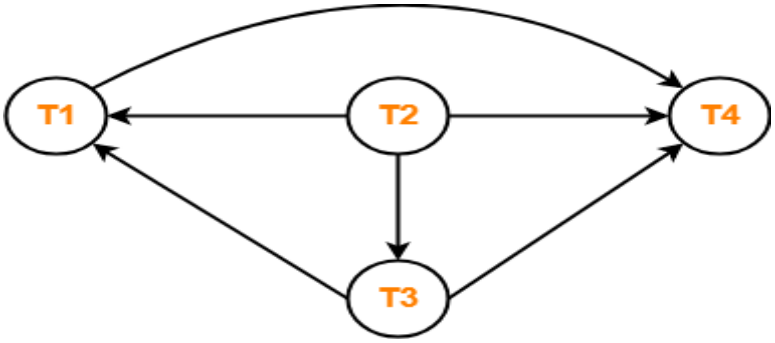
Step-01:

List all the conflicting operations and determine the dependency between the transactions-

- $R_2(X)$, $W_3(X)$ ($T_2 \rightarrow T_3$)
- $R_2(X)$, $W_1(X)$ ($T_2 \rightarrow T_1$)
- $W_3(X)$, $W_1(X)$ ($T_3 \rightarrow T_1$)
- $W_3(X)$, $R_4(X)$ ($T_3 \rightarrow T_4$)
- $W_1(X)$, $R_4(X)$ ($T_1 \rightarrow T_4$)
- $W_2(Y)$, $R_4(Y)$ ($T_2 \rightarrow T_4$)

Step-02:

Draw the precedence graph-



- Clearly, there exists no cycle in the precedence graph.
- Therefore, the given schedule S is conflict serializable.

Checking Whether S is Recoverable Or Not-

Conflict serializable schedules are always recoverable.Therefore, the given schedule S is recoverable.

Problem-03:

Check whether the given schedule S is conflict serializable or not. If yes, then determine all the possible serialized schedules-

T1	T2	T3	T4
			R(A)
	R(A)		
		R(A)	
W(B)			
	W(A)		
		R(B)	
	W(B)		

Solution-

Checking Whether S is Conflict Serializable Or Not-

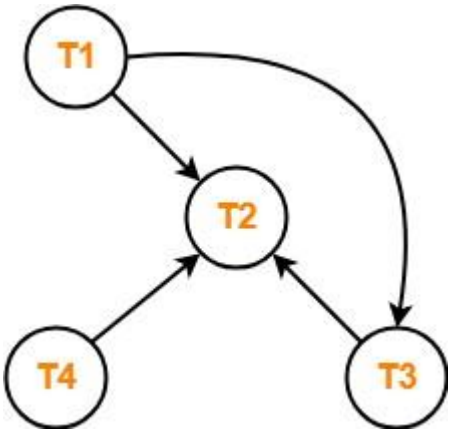
Step-01:

List all the conflicting operations and determine the dependency between the transactions-

- $R_4(A)$, $W_2(A)$ ($T_4 \rightarrow T_2$)
- $R_3(A)$, $W_2(A)$ ($T_3 \rightarrow T_2$)
- $W_1(B)$, $R_3(B)$ ($T_1 \rightarrow T_3$)
- $W_1(B)$, $W_2(B)$ ($T_1 \rightarrow T_2$)
- $R_3(B)$, $W_2(B)$ ($T_3 \rightarrow T_2$)

Step-02:

Draw the precedence graph-



- Clearly, there exists no cycle in the precedence graph.
- Therefore, the given schedule S is conflict serializable.

Finding the Serialized Schedules-

All the possible topological orderings of the above precedence graph will be the possible serialized schedules.

- The topological orderings can be found by performing the **Topological Sort** of the above precedence graph.
  - Check in-degree of each vertex
  - Vertex has the least in-degree.
  - So, remove vertex and its associated edges.
  - Now, update the in-degree of other vertices.

After performing the topological sort, the possible serialized schedules are-

1. $T_1 \rightarrow T_3 \rightarrow T_4 \rightarrow T_2$
2. $T_1 \rightarrow T_4 \rightarrow T_3 \rightarrow T_2$
3. $T_4 \rightarrow T_1 \rightarrow T_3 \rightarrow T_2$

Problem-04:

Determine all the possible serialized schedules for the given schedule-

T1	T2
R(A)	
A = A-10	
	R(A)
	Temp = 0.2 x A
	W(A)
	R(B)
W(A)	
R(B)	
B = B+10	
W(B)	
	B = B+Temp
	W(B)

Solution-

The given schedule S can be rewritten as-

T1	T2
R(A)	
	R(A)
	W(A)
	R(B)
W(A)	
R(B)	
W(B)	
	W(B)

This is because we are only concerned about the read and write operations taking place on the database.

Checking Whether S is Conflict Serializable Or Not-

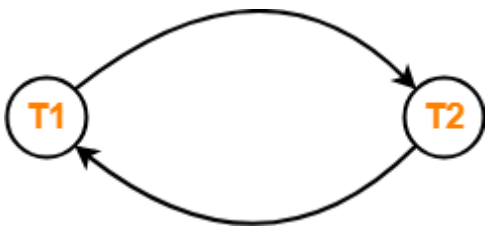
Step-01:

List all the conflicting operations and determine the dependency between the transactions-

- $R_1(A)$, $W_2(A)$ $(T_1 \rightarrow T_2)$
- $R_2(A)$, $W_1(A)$ $(T_2 \rightarrow T_1)$
- $W_2(A)$, $W_1(A)$ $(T_2 \rightarrow T_1)$
- $R_2(B)$, $W_1(B)$ $(T_2 \rightarrow T_1)$
- $R_1(B)$, $W_2(B)$ $(T_1 \rightarrow T_2)$
- $W_1(B)$, $W_2(B)$ $(T_1 \rightarrow T_2)$

Step-02:

Draw the precedence graph-



- Clearly, there exists a cycle in the precedence graph.
- Therefore, the given schedule S is not conflict serializable.
- Thus, Number of possible serialized schedules = 0.

View Serializability

A schedule is viewed serializable if it is view equivalent to a serial schedule. If a schedule is conflict serializable, then it will be view serializable. The view serializable which does not conflict with serializable, contains blind writes.

Non-Serial S1		Serial S2	
Transaction T1	Transaction T2	Transaction T1	Transaction T2
R(X)		R(X)	
W(X)		W(X)	
	R(X)	R(Y)	
	W(X)	W(Y)	
R(Y)			R(X)
W(Y)			W(X)
	R(Y)		R(Y)
	W(Y)		W(Y)

S2 is serial schedule of s1.if we can prove that they are view equivalent then we can say that given schedule S1 is view serializable

If a given schedule is found to be view equivalent to some serial schedule, then it is called as a view serializable schedule.

View Equivalent Schedules-

Consider two schedules S1 and S2 each consisting of two transactions T1 and T2. Schedules S1 and S2 are called view equivalent if the following three conditions hold true for them-

Condition-01: 1. Initial Read

For each data item X, if transaction T_i reads X from the database initially in schedule S1, then in schedule S2 also, T_i must perform the initial read of X from the database.

"Initial readers must be same for all the data items".

An initial read of both schedules must be the same. Suppose two schedule S1 and S2. In schedule S1, if a transaction T1 is reading the data item A, then in S2, transaction T1 should also read A.

T1	T2
Read(A)	
	Write(A)

Schedule S1

T1	T2
	Write(A)
Read(A)	

Schedule S2

Above two schedules are view equivalent because Initial read operation in S1 is done by T1 and in S2 it is also done by T1.

Condition-02: .Updated Read

If transaction T_i reads a data item that has been updated by the transaction T_j in schedule S1, then in schedule S2 also, transaction T_i must read the same data item that has been updated by the transaction T_j.

"Write-read sequence must be same."

T1	T2	T3
Write(A)	Write(A)	Read(A)

Schedule S1

T1	T2	T3
Write(A)	Write(A)	Read(A)

Schedule S2

Above two schedules are not view equal because, in S1, T3 is reading A updated by T2 and in S2, T3 is reading A updated by T1.

Condition-03: Final Write

For each data item X, if X has been updated at last by transaction T_i in schedule S1, then in schedule S2 also, X must be updated at last by transaction T_i.

"Final writers must be same for all the data items".

A final write must be the same between both the schedules. In schedule S1, if a transaction T1 updates A at last then in S2, final writes operations should also be done by T1.

T1	T2	T3
Write(A)	Read(A)	Write(A)

Schedule S1

T1	T2	T3
Write(A)	Read(A)	Write(A)

Schedule S2

Above two schedules is view equal because Final write operation in S1 is done by T3 and in S2, the final write operation is also done by T3.

Checking Whether a Schedule is View Serializable Or Not-

Method-01:

Check whether the given schedule is conflict serializable or not.

- If the given schedule is conflict serializable, then it is surely view serializable. Stop and report your answer.
- If the given schedule is not conflict serializable, then it may or may not be view serializable. Go and check using other methods.

Method-02:

Check if there exists any blind write operation**.(Writing without reading is called as a blind write).**

- If there does not exist any blind write, then the schedule is surely not view serializable. Stop and report your answer.
- If there exists any blind write, then the schedule may or may not be view serializable. Go and check using other methods.

No blind write means not a view serializable schedule.

Method-03:

In this method, try finding a view equivalent serial schedule.

- By using the above three conditions, write all the dependencies.
- Then, draw a graph using those dependencies.
- If there exists no cycle in the graph, then the schedule is view serializable otherwise not.

Example 1:

T1	T2	T3
Read(A) Write(A)	Write(A)	Write(A)

Schedule S

With 3 transactions, the total number of possible schedule

1. = 3! = 6
2. S1 = <T1 T2 T3>
3. S2 = <T1 T3 T2>
4. S3 = <T2 T3 T1>
5. S4 = <T2 T1 T3>
6. S5 = <T3 T1 T2>
7. S6 = <T3 T2 T1>

Taking first schedule S1:

T1	T2	T3
Read(A) Write(A)	Write(A)	Write(A)

Schedule S1

Step 1: final updation on data items

In both schedules S and S1, there is no read except the initial read that's why we don't need to check that condition.

Step 2: Initial Read

The initial read operation in S is done by T1 and in S1, it is also done by T1.

Step 3: Final Write
The final write operation in S is done by T3 and in S1, it is also done by T3. So, S and S1 are view Equivalent.
The first schedule S1 satisfies all three conditions, so we don't need to check another schedule.
Hence, view equivalent serial schedule is:
T1 → T2 → T3

Problem-02:

Check whether the given schedule S is view serializable or not-

T1	T2	T3	T4
R (A)			
	R (A)		
		R (A)	
			R (A)
W (B)			
	W (B)		
		W (B)	
			W (B)

Solution-

We know, if a schedule is conflict serializable, then it is surely view serializable.So, let us check whether the given schedule is conflict serializable or not.

Checking Whether S is Conflict Serializable Or Not-

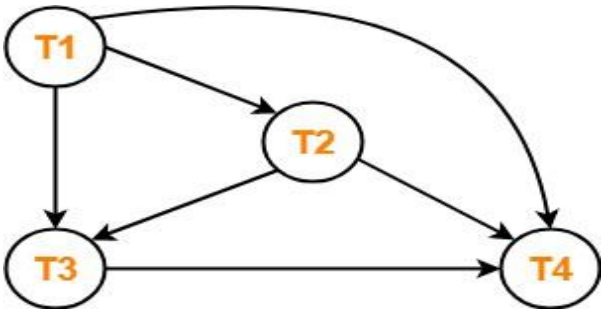
Step-01:

List all the conflicting operations and determine the dependency between the transactions-

- $W_1(B)$, $W_2(B)$ $(T_1 \rightarrow T_2)$
- $W_1(B)$, $W_3(B)$ $(T_1 \rightarrow T_3)$
- $W_1(B)$, $W_4(B)$ $(T_1 \rightarrow T_4)$
- $W_2(B)$, $W_3(B)$ $(T_2 \rightarrow T_3)$
- $W_2(B)$, $W_4(B)$ $(T_2 \rightarrow T_4)$
- $W_3(B)$, $W_4(B)$ $(T_3 \rightarrow T_4)$

Step-02:

Draw the precedence graph-



- Clearly, there exists no cycle in the precedence graph.
- Therefore, the given schedule S is conflict serializable.
- Thus, we conclude that the given schedule is also view serializable.

Problem-02:

Check whether the given schedule S is view serializable or not-

T1	T2	T3
R (A)		
	R (A)	
		W (A)
W (A)		

Solution-

We know, if a schedule is conflict serializable, then it is surely view serializable.So, let us check whether the given schedule is conflict serializable or not.

Checking Whether S is Conflict Serializable Or Not-

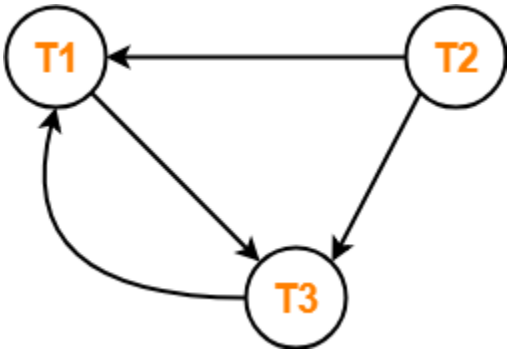
Step-01:

List all the conflicting operations and determine the dependency between the transactions-

- $R_1(A)$, $W_3(A)$ $(T_1 \rightarrow T_3)$
- $R_2(A)$, $W_3(A)$ $(T_2 \rightarrow T_3)$
- $R_2(A)$, $W_1(A)$ $(T_2 \rightarrow T_1)$
- $W_3(A)$, $W_1(A)$ $(T_3 \rightarrow T_1)$

Step-02:

Draw the precedence graph-



Clearly, there exists a cycle in the precedence graph.

- Therefore, the given schedule S is not conflict serializable.
- Since, the given schedule S is not conflict serializable, so, it may or may not be view serializable.
- To check whether S is view serializable or not, let us use another method.
- Let us check for blind writes.

Checking for Blind Writes-

There exists a blind write $W_3(A)$ in the given schedule S.Therefore, the given schedule S may or may not be view serializable.

Now,

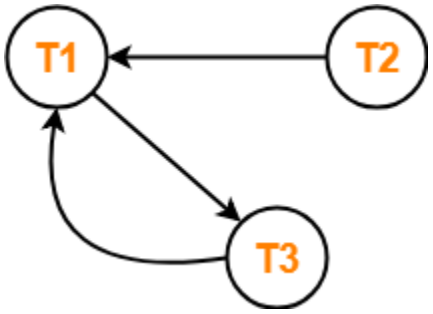
- To check whether S is view serializable or not, let us use another method.

Let us derive the dependencies and then draw a dependency graph.

Drawing a Dependency Graph-

- T1 firstly reads A and T3 firstly updates A.
- So, T1 must execute before T3.
- Thus, we get the dependency **T1 → T3**.
- Final updation on A is made by the transaction T1.
- So, T1 must execute after all other transactions.
- Thus, we get the dependency **(T2, T3) → T1**.
- There exists no write-read sequence.

Now, let us draw a dependency graph using these dependencies-



Clearly, there exists a cycle in the dependency graph.

- Thus, we conclude that the given schedule S is not view serializable.

Problem-03:

Check whether the given schedule S is view serializable or not-

T1	T2
R (A)	
A = A + 10	
	R (A)
	A = A + 10
W (A)	
	W (A)
R (B)	
B = B + 20	
	R (B)
	B = B x 1.1
W (B)	
	W (B)

Solution-

We know, if a schedule is conflict serializable, then it is surely view serializable.So, let us check whether the given schedule is conflict serializable or not.

Checking Whether S is Conflict Serializable Or Not-

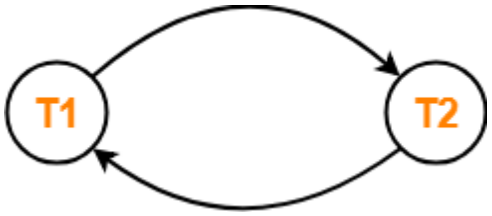
Step-01:

List all the conflicting operations and determine the dependency between the transactions-

- $R_1(A)$, $W_2(A)$ $(T_1 \rightarrow T_2)$
- $R_2(A)$, $W_1(A)$ $(T_2 \rightarrow T_1)$
- $W_1(A)$, $W_2(A)$ $(T_1 \rightarrow T_2)$
- $R_1(B)$, $W_2(B)$ $(T_1 \rightarrow T_2)$
- $R_2(B)$, $W_1(B)$ $(T_2 \rightarrow T_1)$

Step-02:

Draw the precedence graph-



- Clearly, there exists a cycle in the precedence graph.
- Therefore, the given schedule S is not conflict serializable.
- Since, the given schedule S is not conflict serializable, so, it may or may not be view serializable.
- To check whether S is view serializable or not, let us use another method.
- Let us check for blind writes.

Checking for Blind Writes-

There exists no blind write in the given schedule S.Therefore, it is surely not view serializable.

Alternatively,

- You could directly declare that the given schedule S is not view serializable.
- This is because there exists no blind write in the schedule.
- You need not check for conflict serializability.

Problem-04:

Check whether the given schedule S is view serializable or not. If yes, then give the serial schedule.

S : $R_1(A)$, $W_2(A)$, $R_3(A)$, $W_1(A)$, $W_3(A)$

Solution-

For simplicity and better understanding, we can represent the given schedule pictorially as-

T1	T2	T3
R (A)		
	W (A)	
		R (A)
W (A)		
		W (A)

We know, if a schedule is conflict serializable, then it is surely view serializable.So, let us check whether the given schedule is conflict serializable or not.

Checking Whether S is Conflict Serializable Or Not-

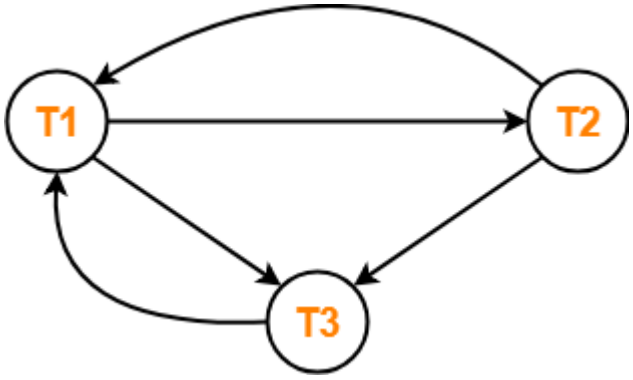
Step-01:

List all the conflicting operations and determine the dependency between the transactions-

- $R_1(A)$, $W_2(A)$ $(T_1 \rightarrow T_2)$
- $R_1(A)$, $W_3(A)$ $(T_1 \rightarrow T_3)$
- $W_2(A)$, $R_3(A)$ $(T_2 \rightarrow T_3)$
- $W_2(A)$, $W_1(A)$ $(T_2 \rightarrow T_1)$
- $W_2(A)$, $W_3(A)$ $(T_2 \rightarrow T_3)$
- $R_3(A)$, $W_1(A)$ $(T_3 \rightarrow T_1)$
- $W_1(A)$, $W_3(A)$ $(T_1 \rightarrow T_3)$

Step-02:

Draw the precedence graph-



- Clearly, there exists a cycle in the precedence graph.
- Therefore, the given schedule S is not conflict serializable.

Now,

- Since, the given schedule S is not conflict serializable, so, it may or may not be view serializable.
- To check whether S is view serializable or not, let us use another method.
- Let us check for blind writes.

Checking for Blind Writes-

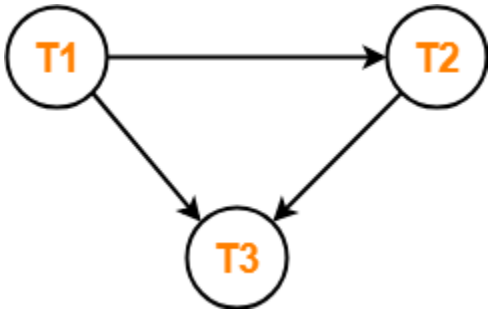
There exists a blind write W_2 (A) in the given schedule S.Therefore, the given schedule S may or may not be view serializable.

- To check whether S is view serializable or not, let us use another method.
- Let us derive the dependencies and then draw a dependency graph.

Drawing a Dependency Graph-

- T1 firstly reads A and T2 firstly updates A.
- So, T1 must execute before T2.
- Thus, we get the dependency **T1 → T2**.
- Final updation on A is made by the transaction T3.
- So, T3 must execute after all other transactions.
- Thus, we get the dependency **(T1, T2) → T3**.
- From write-read sequence, we get the dependency **T2 → T3**

Now, let us draw a dependency graph using these dependencies-



- Clearly, there exists no cycle in the dependency graph.
- Therefore, the given schedule S is view serializable.
- The serialization order T1 → T2 → T3.

Non-Serializability in DBMS

A non-serial schedule that is not serializable is called a non-serializable schedule. Non-serializable schedules may/may not be consistent or recoverable. Non-serializable Schedule is divided into types:

1. Recoverable Schedule
2. Non-recoverable Schedule

Characteristics-

Non-serializable schedules-

may or may not be consistent
may or may not be recoverable

Irrecoverable Schedules-

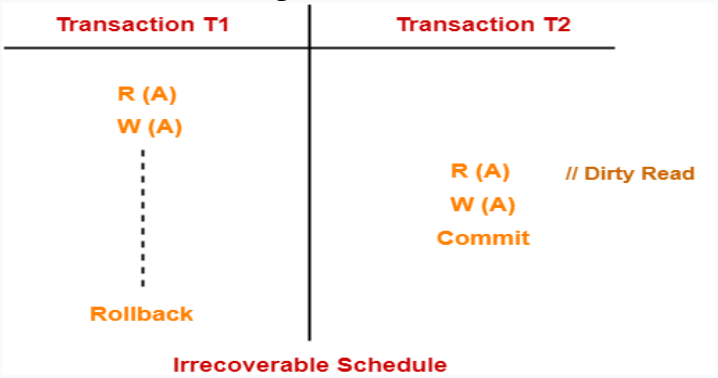
If in a schedule,

- A transaction performs a dirty read operation from an uncommitted transaction
- And commits before the transaction from which it has read the value

then such a schedule is known as an **Irrecoverable Schedule**.

Example-

Consider the following schedule-



Here,

T2 performs a dirty read operation.
T2 commits before T1.
T1 fails later and roll backs.
The value that T2 read now stands to be incorrect.
T2 can not recover since it has already committed.

T1	T1's buffer space	T2	T2's buffer space
Read(A);	A = 6500		
A = A - 500;	A = 6000		
Write(A);	A = 6000		
		Read(A);	A = 6000
		A =A + 1000;	A = 7000
		Write(A);	A = 7000
		Commit;	
Failure Point			
Commit;			

The above table shows a schedule which has two transactions. T1 reads and writes the value of A and that value is read and written by T2. T2 commits but later on, T1 fails. Due to the failure, we have to rollback T1. T2 should also be rollback because it reads the value written by T1, but T2 can't be rollback because it already committed. So this type of schedule is known as irrecoverable schedule.

Recoverable Schedules-

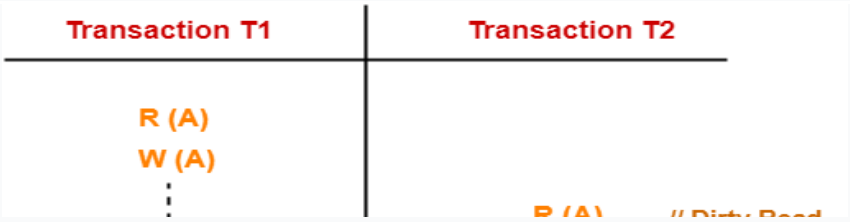
If in a schedule,

A transaction performs a dirty read operation from an uncommitted transaction
And its commit operation is delayed till the uncommitted transaction either commits or roll backs then such a schedule is known as a **Recoverable Schedule**.
The commit operation of the transaction that performs the dirty read is delayed.

This ensures that it still has a chance to recover if the uncommitted transaction fails later.

Example-

Consider the following schedule-



Dirty read
Uncommitted value not original value from database taken from another transaction

T2 performs a dirty read operation.

The commit operation of T2 is delayed till T1 commits or roll backs.

T1 commits later.

T2 is now allowed to commit.

In case, T1 would have failed, T2 has a chance to recover by rolling back.

T1	T1's buffer space	T2	T2's buffer space	Database
				A = 6500
Read(A);	A = 6500			A = 6500
A = A - 500;	A = 6000			A = 6500
Write(A);	A = 6000			A = 6000
		Read(A);	A = 6000	A = 6000
		A =A + 1000;	A = 7000	A = 6000
		Write(A);	A = 7000	A = 7000
Failure Point				
Commit;				
		Commit;		

The above table shows a schedule with two transactions. Transaction T1 reads and writes A, and that value is read and written by transaction T2. But later on, T1 fails. Due to this, we have to rollback T1. T2 should be rollback because T2 has read the value written by T1. As it has not committed before T1 commits so we can rollback transaction T2 as well. So it is recoverable with cascade rollback.

Checking Whether a Schedule is Recoverable or Irrecoverable-

Method-01:

Check whether the given schedule is conflict serializable or not.

If the given schedule is conflict serializable, then it is surely recoverable. Stop and report your answer.

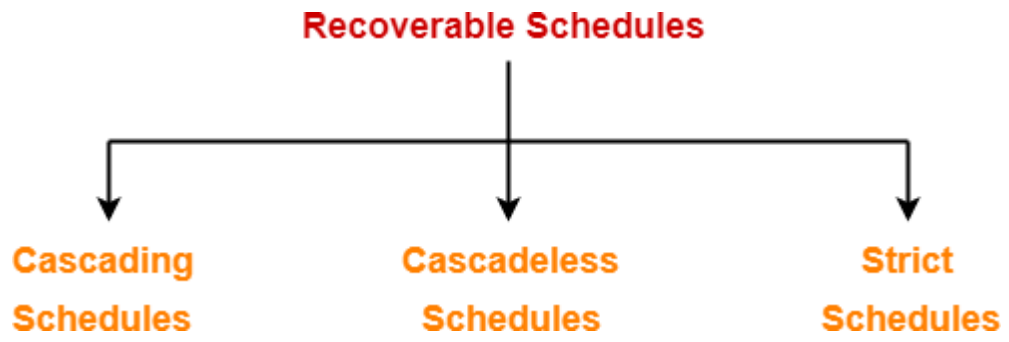
If the given schedule is not conflict serializable, then it may or may not be recoverable. Go and check using other methods.

- All conflict serializable schedules are recoverable.
- All recoverable schedules may or may not be conflict serializable.

Method-02:
Check if there exists any dirty read operation.
(Reading from an uncommitted transaction is called as a dirty read)
If there does not exist any dirty read operation, then the schedule is surely recoverable. Stop and report your answer.
If there exists any dirty read operation, then the schedule may or may not be recoverable.
If there exists a dirty read operation, then follow the following cases-

Case-01:
If the commit operation of the transaction performing the dirty read occurs before the commit or abort operation of the transaction which updated the value, then the schedule is irrecoverable.
Case-02:
If the commit operation of the transaction performing the dirty read is delayed till the commit or abort operation of the transaction which updated the value, then the schedule is recoverable.
No dirty read means a recoverable schedule.

A recoverable schedule may be any one of these kinds-



1. Cascading Schedule
2. Cascadeless Schedule
3. Strict Schedule

Cascading Schedule-

If in a schedule, failure of one transaction causes several other dependent transactions to rollback or abort, then such a schedule is called as a **Cascading Schedule** or **Cascading Rollback** or **Cascading Abort**.

- It simply leads to the wastage of CPU time.

S1

Transaction T1	Transaction T2	Transaction T3
R(X)		
W(X)		
	R(X)	
	W(X)	
		R(X)
	Cascading abort	W(X)
a	a	a

Here, Transaction T2 depends on transaction T1, and transaction T3 depends on transaction T2. Thus, in this Schedule, the failure of transaction T1 will cause transaction T2 to roll back, and a similar case for transaction T3. Therefore, it is a cascading schedule. If the transactions T2 and T3 had been committed before the failure of transaction T1, then the Schedule would have been irrecoverable.

Cascadeless Schedule-

If in a schedule, a transaction is not allowed to read a data item until and unless the last transaction that has been written is committed/aborted, then such a schedule is called a Cascadeless Schedule. It avoids cascading rollback and thus saves CPU time. To prevent cascading rollbacks, it disallows a transaction from reading uncommitted changes from another transaction in the same Schedule. In other words, if some transaction Ty wants to read a value that has been updated or written by some other transaction Tx, then only after the commit of Tx, the commit of Ty must read it. Look at the example shown below.

S2

Transaction T1	Transaction T2	Transaction T3
R(X)		
W(X)		
C		
	R(X)	
	W(X)	
	C	
		R(X)
		W(X)
		C

Here, the updated value of X is read by transaction T2 only after the commit of transaction T1. Hence, the Schedule is Cascadeless schedule.

In other words,

- Cascadeless schedule allows only committed read operations.
- Therefore, it avoids cascading roll back and thus saves CPU time.

Example-

NOTE-

Cascadeless schedule allows only committed read operations.

Strict Schedule

If in a schedule, until the last transaction that has written it is committed or aborted, a transaction is neither allowed to read nor write data item, then such a schedule is called as Strict Schedule. Let's say we have two transactions Ta and Tb. The write operation of transaction Ta precedes the read or write operation of transaction Tb, so the commit or abort operation of transaction Ta should also precede the read or write of Tb. A strict Schedule allows only committed read and write operations. This Schedule implements more restrictions than cascadeless schedule. Consider an example shown below.

Transaction (Ta)	Transaction (Tb)
R(X)	
	R(X)
W(X)	
Commit	
	W(X)
	R(X)
	commit

- Here, transaction Tb reads/writes the written value of transaction Ta only after the transaction Ta commits. Hence, the Schedule is a strict Schedule.
- Here the write operation W(X) of Ta precedes the conflicting operation (Read or Write operation) of Tb so the conflicting operation of Tb had to wait the commit operation of Ta.

Remember-

- Strict schedules are more strict than cascadeless schedules.
- All strict schedules are cascadeless schedules.
- All cascadeless schedules are not strict schedules.

Serial Schedules Vs Serializable Schedules-

Serial Schedules	Serializable Schedules
No concurrency is allowed. Thus, all the transactions necessarily execute serially one after the other.	Concurrency is allowed. Thus, multiple transactions can execute concurrently.
Serial schedules lead to less resource utilization and CPU throughput.	Serializable schedules improve both resource utilization and CPU throughput.
Serial Schedules are less efficient as compared to serializable schedules. (due to above reason)	Serializable Schedules are always better than serial schedules. (due to above reason)

Concurrency Problems in DBMS-

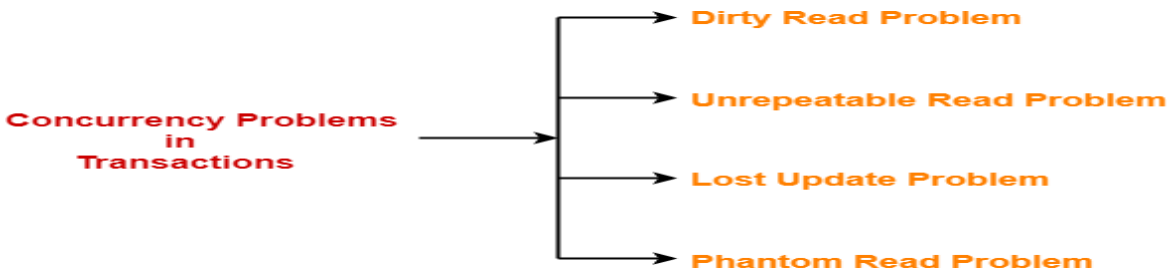
When multiple transactions execute concurrently in an uncontrolled or unrestricted manner, then it might lead to several problems.

- Such problems are called as **concurrency problems**.

Advantages

- The advantages of concurrency control are as follows −
- Waiting time will be decreased.
- Response time will decrease.
- Resource utilization will increase.
- System performance & Efficiency is increased.

The concurrency problems are-



1. Dirty Read Problem
2. Unrepeatable Read Problem
3. Lost Update Problem
4. Phantom Read Problem

1. Dirty Read Problem-

Reading the data written by an uncommitted transaction is called as dirty read.

This read is called as dirty read because-

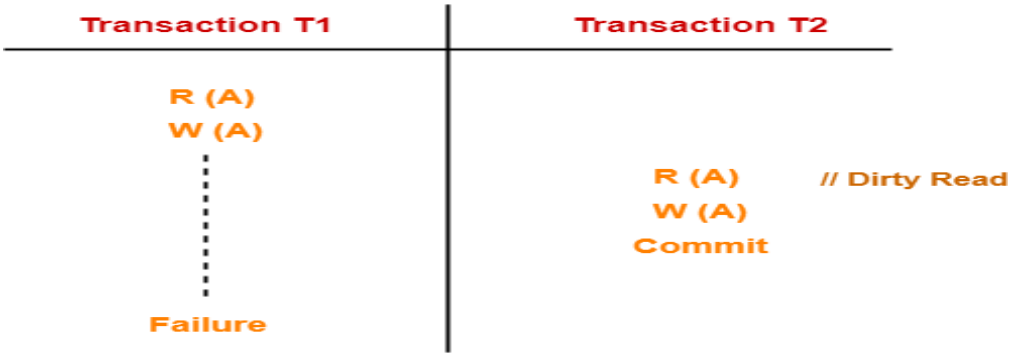
- There is always a chance that the uncommitted transaction might roll back later.
- Thus, uncommitted transaction might make other transactions read a value that does not even exist.
- This leads to inconsistency of the database.

NOTE-

Dirty read does not lead to inconsistency always.

- It becomes problematic only when the uncommitted transaction fails and roll backs later due to some reason.

Example-



Here,

1. T1 reads the value of A=5.
2. T1 updates the value of A in the buffer.
   (increment A by 1,A=6)
3. T2 reads the value of A from the buffer.(value of A=6)
4. T2 writes the updated the value of A.
5. T2 commits.
6. T1 fails in later stages and rolls back.(A=5)

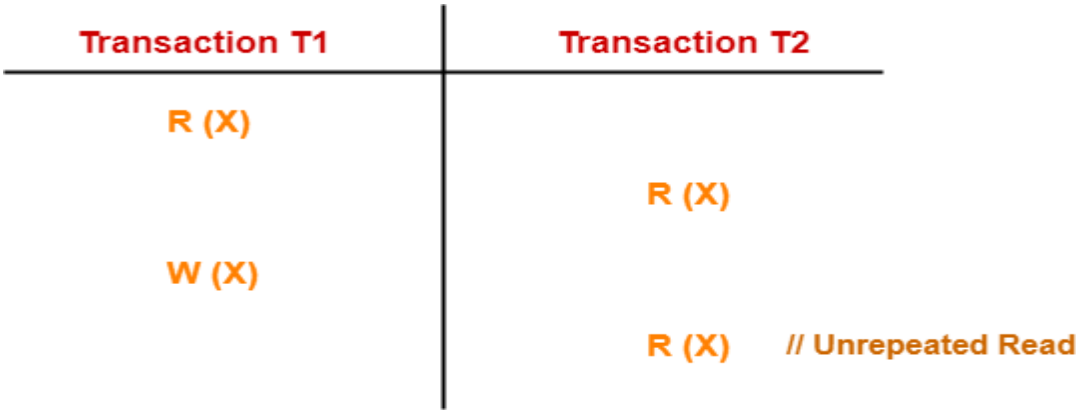
In this example,

- T2 reads the dirty value of A written by the uncommitted transaction T1.
- T1 fails in later stages and roll backs.
- Thus, the value that T2 read now stands to be incorrect.
- Therefore, database becomes inconsistent.

2. Unrepeatable Read Problem-

This problem occurs when a transaction gets to read unrepeated i.e. different values of the same variable in its different read operations even when it has not updated its value.

Example-



Here,

1. T1 reads the value of X (= 10 say).
2. T2 reads the value of X (= 10).
3. T1 updates the value of X (from 10 to 15 say) in the buffer.

4. T2 again reads the value of X (but = 15).

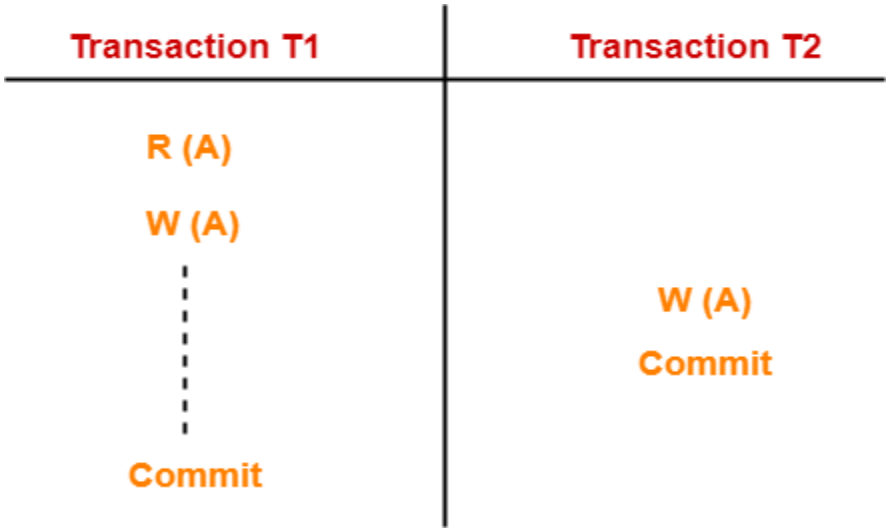
In this example,

- T2 gets to read a different value of X in its second reading.
- T2 wonders how the value of X got changed because according to it, it is running in isolation.

3. Lost Update Problem(W-W conflict)-

This problem occurs when multiple transactions execute concurrently and updates from one or more transactions get lost.

Example-



Here,

1. T1 reads the value of A (= 10 say).
2. T2 updates the value to A (= 15 say) in the buffer.
3. T2 does blind write A = 25 (write without read) in the buffer.
4. T2 commits.
5. When T1 commits, it writes A = 25 in the database.

In this example,

- T1 writes the over written value of X in the database.
- Thus, update from T1 gets lost.

NOTE-

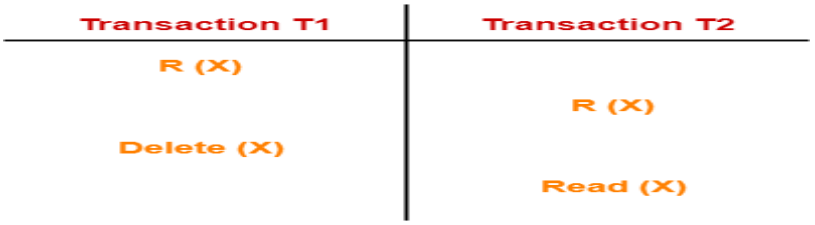
This problem occurs whenever there is a write-write conflict.

- In write-write conflict, there are two writes one by each transaction on the same data item without any read in the middle.

4. Phantom Read Problem-

This problem occurs when a transaction reads some variable from the buffer and when it reads the same variable later, it finds that the variable does not exist.

Example-



Here,

1. T1 reads X.
2. T2 reads X.
3. T1 deletes X.
4. T2 tries reading X but does not find it.

In this example,

- T2 finds that there does not exist any variable X when it tries reading X again.
- T2 wonders who deleted the variable X because according to it, it is running in isolation.

Incorrect Summary Problem

The Incorrect summary problem occurs when there is an incorrect sum of the two data. This happens when a transaction tries to sum two data using an aggregate function and the value of any one of the data get changed by another transaction.

Example: Consider two transactions A and B performing read/write operations on two data DT1 and DT2 in the database DB. The current value of DT1 is 1000 and DT2 is 2000: The following table shows the read/write operations in A and B transactions.

Time	A	B
T1	READ(DT1)	------
T2	add=0	------
T3	add=add+DT1	------
T4	------	READ(DT2)
T5	------	DT2=DT2+500
T6	READ(DT2)	------
T7	add=add+DT2	------

Transaction A reads the value of DT1 as 1000. It uses an aggregate function SUM which calculates the sum of two data DT1 and DT2 in variable add but in between the value of DT2 get changed from 2000 to 2500 by transaction B. Variable add uses the modified value of DT2 and gives the resultant sum as 3500 instead of 3000.

7. Concurrency Control Techniques: Concurrency Control Protocols

To avoid concurrency control problems and to maintain consistency and serializability during the execution of concurrent transactions some rules are made. These rules are known as Concurrency Control Protocols.

Lock Based Concurrency Control Protocol

In this type of protocol, any transaction cannot read or write data until it acquires an appropriate lock on it. There are **two types** of lock

1. Shared Lock

- It is also known as a Read-only lock. In a shared lock, if any transaction wants to perform Read operation on data then it must acquire the shared lock on data first.

Transaction T1	
LOCK S (A)	// Shared Lock on data "A"
R (A)	// Read Operation on data "A"
Unlock (A)	// Unlock data "A"

- If a transaction (say T1) holds an shared lock on Data(say A) and some other transaction (say T2) want to perform read operation on same data (A), then T2 can also acquire the shared lock without waiting the unlocking of T1. It is because Read-Read is not conflict.

2. Exclusive Lock

In an exclusive lock, if any transaction want to perform Read and Write both operation on same data then it must acquire the exclusive lock first.

Transaction T1	
LOCK S (A)	// Exclusive Lock on data "A"
R (A)	// Read Operation on data "A"
W (A)	// Write Operation on data "A"
Unlock (A)	// Unlock data "A"

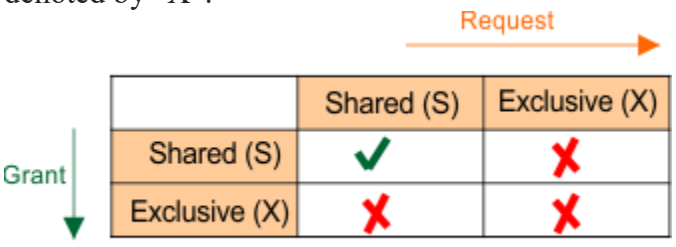
- In exclusive lock, multiple transactions do not perform the Write operations on same data simultaneously.
- if a transaction (say T1) holds an exclusive lock on Data(say A) and some other transaction (say T2) want to access the same data (A), then it T2 has to wait until T1 unlock the exclusive lock.

Note: Shared and exclusive lock on different data items can be applied any times without any problem

Compatibility Lock Table

Following compatibly table is used when multiple transaction wants to perform read or write operation on same data items.

Suppose T1 and T2 are parallel transaction and both wants to perform read and write operation on same data say "A". Shared lock is denoted by "S" and Exclusive lock is denoted by "X".



Compatibility Lock Table

Case 01: (Shared to shared): If T1 has a Shared lock on data (A) then we allow to T2 to shared-lock on same data (A) without unlocking data of T1.

Explanation: When a transaction T1 holds a shared lock for read data and T2 wants the shared lock for read operation on same data "A" then shared lock is granted because Read-Read is not a conflict.

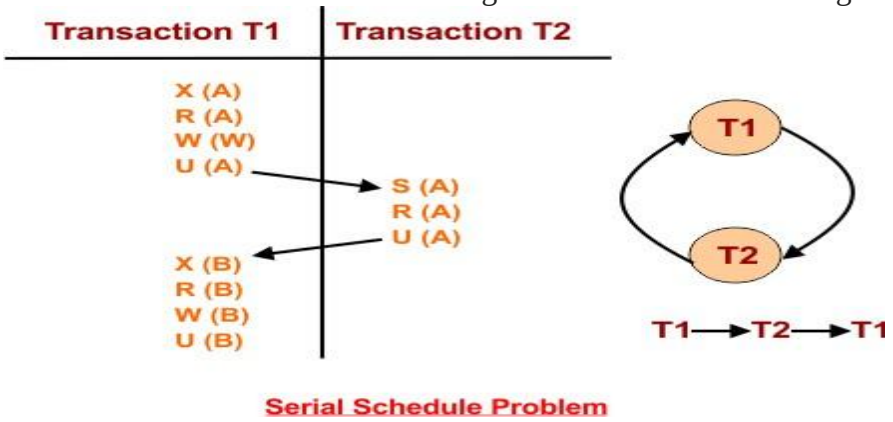
Case 02: Case (Shared to Exclusive) If T1 has an Shared- lock on data (A) then we cannot allow to T2 to exclusive-lock on same data (A) until unless T1 unlock the data (A)
Explanation: When a transaction T1 holds a shared lock for read data ("A") and T2 wants the Exclusive lock for read and write operation on same data "A" then shared lock is not granted because Read-Write is a conflict.

Case 03: Case (Exclusive to shared) If T1 has an exclusive lock on data (A) then we cannot allow to T2 to lock either shared or Exclusive on same data(A) until unless T1 unlock the data(A)
Explanation: When a transaction T1 holds a Exclusive lock for read and write data ("A") and T2 wants the Exclusive lock for read and write operation on same data "A" then Exclusive lock is not granted because Read-Write and Write-Write is a conflict.

Case 04: (Exclusive to Exclusive) If T1 has an exclusive lock on data (A) then we cannot allow to T2 to lock Exclusive on same data(A) until unless T1 unlock the data(A)
Explanation: When a transaction T1 holds a Exclusive lock for read and write data ("A") and T2 wants the Exclusive lock for read and write operation on same data "A" then Exclusive lock is not granted because Read-Write and Write-Write is a conflict there.

All above conditions are required when multiple transaction are using the same data. If T1 has exclusive-lock on data A then T2 can exclusive lock on B without waiting the unlocking of T1 on data A concurrently.

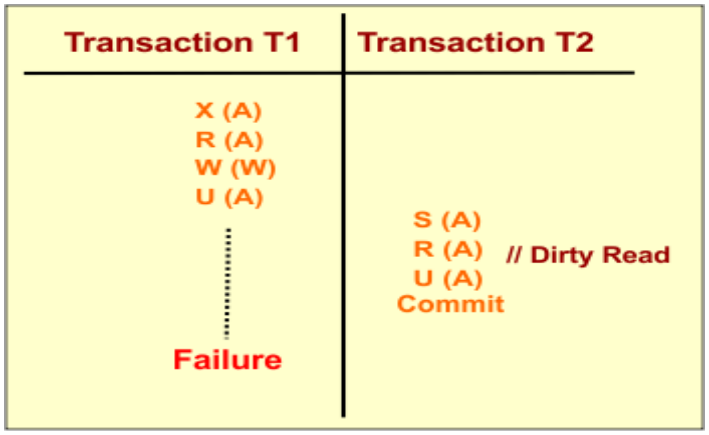
Problems in Shared-Exclusive Locking
When the Shared-Exclusive locking is given properly then there may still exist the following problem
Problem-1: Produced schedule through Shared-Exclusive locking is not a serial always.



Serial Schedule Problem

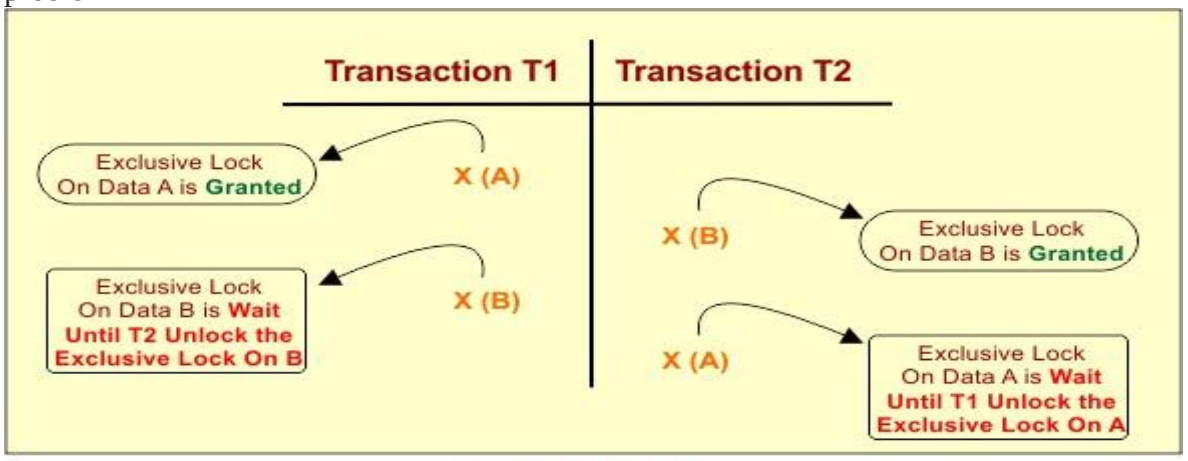
Explanation: See the above example where read/write locking is given properly but still a loop present in the schedule of T1 and T2. We know if a loop is there then it may or may not be a serial

Problem-2: Produced schedule through Shared-Exclusive locking may be irrecoverable



Irrecoverable Problem

Problem-3: Produced schedule through Shared-Exclusive locking may contains a deadlock problem



Deadlock Problem

Problem-4: Produced schedule through Shared-Exclusive locking may contains still starvation

Time	T1	T2	T3	T4
Time1		S (A)		
Time2	X (A) Wait			
Time3			S (A)	
Time4		U (A)		
Time5				S (A)
Time6			U (A)	
Time7				U (A)
Time8	X (A) Granted			

Starvation Problem

Explanation

- T2 request for shared-lock on data "A" which is granted directly , Now let suppose T1 request for exclusive lock on data "A" which will not be granted until T2 unlock the data A.
- Suppose as T2 was unlocking data A, T3 also acquire Shared-lock on same data A. It is possible shared lock and shared lock at a time on same data. So, T1 has to wait until T3 unlock the A.

- Suppose as T3 was unlocking data A, T4 also get the Shared-lock on same data A. Now, T1 has to wait until T4 unlock the A.
- So, Transaction T1 wait from time 2 to time 8 for getting exclusive lock on data "A". Therefore, It is a starvation case.
-

Problem-5: Produced schedule through Shared-Exclusive locking may contains still cascading Rollback problem

Time	T1	T2	T3
Time1	S (A)		
Time2	R (A)		
Time3	W (A)		
Time4	U (A)		
Time5		S (A)	
Time6		R (A)	
Time7			S (A)
Time8			R (A)
	Failure	Rollback	

Rollback

Cascading Rollback Problem

Explanation: If the rollback of one transaction cause the rollback of other dependent transactions called cascading rollback. To remove all said problems we use 2-PL

1. Two Phase Locking (2PL) Protocol

- 2-PL is an extension of Shared/Exclusive locking.
- It is used to reduce the problems of Shared/Exclusive locking
- Any schedule which is following 2PL will always serializable which was not in Shared/Exclusive locking

Phases of 2PL

There are two basic phases of two phase locking which are explained below

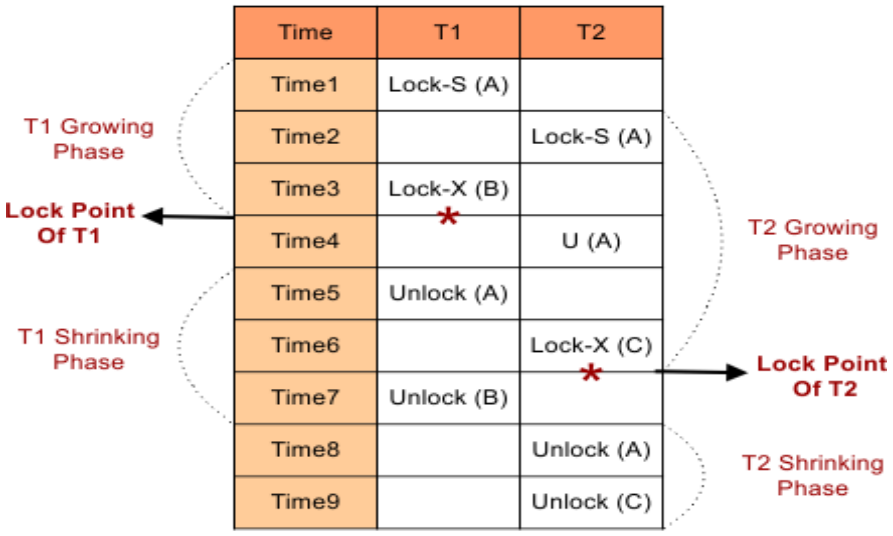
I. Growing Phase: In Growing phase, only Locks are acquired by a transaction and no locks are released by a transaction at that time.

II. Shrinking Phase: In shrinking phase only Locks are released by transaction and no locks are acquired by a transaction at that time.

Important in 2PL is **Lock Point**

Lock Point: The Point at which the growing phase ends (i.e., when transaction takes the final lock) is called Lock Point.

Example: The following diagrams shows growing and shrinking phases in 2-PL.



2-PL Locking Example

Transaction T1

- **Growing Phase:** From Time 1 to 3.
- **Shrinking Phase:** From Time 5 to 7
- **Lock Point:** At time 3

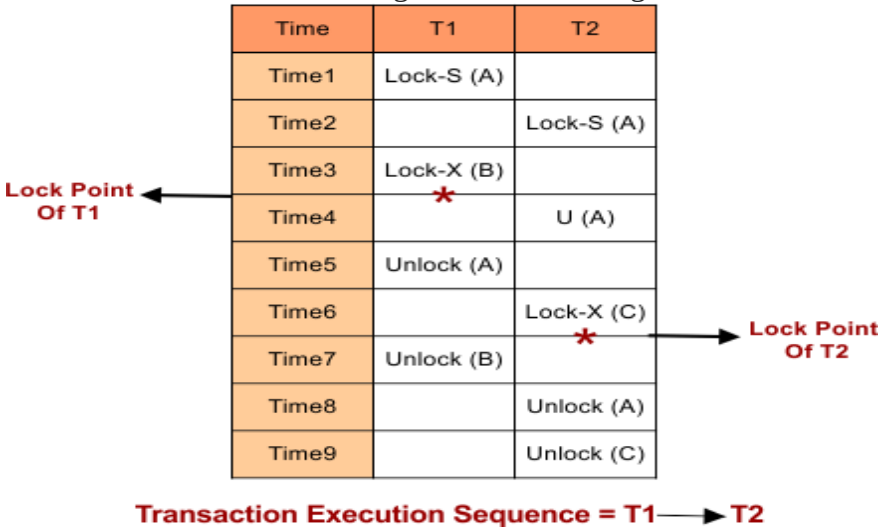
Transaction T2

- **Growing Phase:** From Time 2 to 6
- **Shrinking Phase:** From Time 8 to 9
- **Lock Point:** At Time 6

2PL Transaction Execution Sequence

As we say, 2PL will generate the serial schedule then what will be the sequence of transaction execution?

Answer: If locking point of Transaction T1 comes earlier than transaction T2 then the T1 will execute first than T2. As given below in diagram



Important: when multiple transactions needs to acquire the shared or exclusive lock then it must follow the lock compatibility table.

In the following schedule where T1 holds the shared lock on "A" and T2 also required exclusive lock on "A". This case is not a possible according to lock compatibility table. So, T2 will be blocked(waiting state) until T1 release the the exclusive lock on "A".

Advantages of 2-PL

2PL always ensure the Serilizability. It means the schedule in 2PL must be serial. No loop existing in transaction execution sequence graph.

Disadvantages of 2-PL

1. Produced schedule through 2PL locking may be irrecoverable
2. Produced schedule through 2PL locking may contains a deadlock problem
3. Produced schedule through 2PL locking may contains a Starvation Problem
4. Cascading Rollback Problem

If the rollback of one transaction cause the rollback of other dependent transactions called cascading rollback.

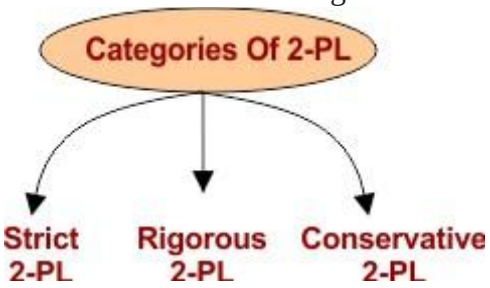
As 2PL remove the Irrecoverability problem but still cascading, deadlock and starvation problem is there. To remove such problems we use different types of 2PL.

Categories of Two Phase Locking

As we know Basic 2-PL achieve the serializability, but if we want to achieve cascadless, recoverability and deadlock removal from schedule then we have to use categories of two phase locking.

Categories of 2-PL

There are three basic categories of 2-PL



Strict Two-Phase Locking Protocol

In DBMS, Cascaded rollbacks are avoided with the concept of a Strict Two-Phase Locking Protocol. A schedule will be in Strict 2PL if It must satisfy the basic 2-PL.

And each transaction should holds all **Exclusive(X) Locks** until the Transaction is Commits or abort.

It is responsible for assuring that if 1 transaction modifies data, there can be no other transaction that will be able to read it until the first transaction commits. The majority of database systems use a strict two-phase locking protocol.

Starvation and Deadlock

When a transaction must wait an unlimited period for a lock, it is referred to as starvation. The following are the causes of starvation :

1. When the locked item waiting scheme is not correctly controlled.
2. When a resource leak occurs.

3. The same transaction is repeatedly chosen as a victim.

Let's know how starvation can be prevented. Random process selection for resource or processor allocation should be avoided since it encourages hunger. The resource allocation priority scheme should contain ideas like aging, in which a process' priority rises as it waits longer. This prevents starvation.

T1	T2
Lock-S(A)	
READ (A)	
WRITE (A)	Lock-S(A) // Granted
LOCK-X(B)	
READ (B)	
WRITE (B)	
Commit (A)	
Commit (B)	
Unlock (A)	
Unlock (B)	

Strict 2-PL

2. Rigorous 2-PL

It is similar to Strict 2-PL in its advantage and disadvantage but little bit more strict than Strict 2-PL.

A schedule will be in Rigorous 2PL if

- It must satisfy the basic 2-PL.
- And each transaction should holds **all Exclusive(X) Locks** as well as **Shared(S) Locks** until the Transaction is Commits or abort.

T1	T2
Lock-S(A)	
READ (A)	
WRITE (A)	Lock-S(A) // Wait
LOCK-X(B)	
READ (B)	
WRITE (B)	
Commit (A)	
Commit (B)	
Unlock (A)	
Unlock (B)	Lock-S(A) // Granted
	READ (A)
	WRITE (A)
	Commit (A)
	Unlock (A)

Rigorous 2-PL

Rigorous 2-PL necessitates the release of any Exclusive(X) and Shared(S) locks held by the transaction until after the Transaction Commits, in addition to the Lock being 2-Phase. Our schedule will be Recoverable and Cascadeless if we follow Rigorous 2-PL.

As a result, it frees us from Cascading Abort, which was still present in Basic 2-PL, and also guarantees Strict Schedules, though Deadlocks are still possible!

2PL + all locks (shared or exclusive) should be held till transaction commit or rollback.

Difference between Strict and Rigorous 2PL

In Strict 2PL, If T1 acquire Shared lock on some data (Say A) and T2 also request for shared lock on same data ("A") then it is granted.

But in Rigorous 2PL, If T1 acquire Shared lock on data "A" and T2 also request for shared lock on same data "A" then it will not be granted. Because T1 has to commit before to grant shared lock to T2.

Following figure explain all

T1	T2
Lock-S(A)	
READ (A)	
WRITE (A)	Lock-S(A) // Granted
LOCK-X(B)	
READ (B)	
WRITE (B)	
Commit (A)	
Commit (B)	
Unlock (A)	
Unlock (B)	

Strict 2-PL

T1	T2
Lock-S(A)	
READ (A)	
WRITE (A)	Lock-S(A) // Wait
LOCK-X(B)	
READ (B)	
WRITE (B)	
Commit (A)	
Commit (B)	
Unlock (A)	
Unlock (B)	Lock-S(A) // Granted
	READ (A)
	WRITE (A)
	Commit (A)
	Unlock (A)

Rigorous 2-PL

Advantages of Strict/ Rigorous 2PL

There are same advantages and disadvantage of Strict/ Rigorous 2PL. Explained under

Recoverable: As T2 can't acquire any lock until T1 committed or abort. So if T1 fail then no effect on the T2 so schedule will become recoverable.

Cascadeless: As T2 can't acquire any lock until T1 committed or abort. So if T1 fail then no effect on the T2 so schedule will become cascadless. Cascadless means rollback of one transaction doesn't effect the other transactions.

Note: Still **deadlock and starvation problems** are exist in strict 2-PL

3. Conservative 2-PL

It is just a assuming technique which cannot implemented where,

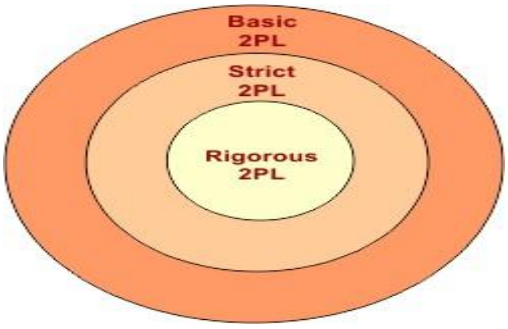
Before to start the transaction, the transaction (i.e. T1) should hold all the locks on all data items(i.e. A, B, C etc). In this way rest of transactions (T2, T3 … Tn) will not access the data until T1 commit or abort.

Conservative 2PL prevents deadlocks along with recoverability and cascadless. Because when any transaction holds all the resources then it will never go to deadlock state.

It is difficult to use in practice because of need to pre-declare the read-set and the write-set which is not possible in many situations

.Compression of 2PL Categories

Rigorous is more strict than strict 2PL and basic 2PL. Look at the following diagram



8. Deadlock Handling

A deadlock is a condition where two or more transactions are waiting indefinitely for one another to give up locks. Deadlock is said to be one of the most feared complications in DBMS as no task ever gets finished and is in waiting for state forever.

For example: In the student table, transaction T1 holds a lock on some rows and needs to update some rows in the grade table. Simultaneously, transaction T2 holds locks on some rows in the grade table and needs to update the rows in the Student table held by Transaction T1.

Now, the main problem arises. Now Transaction T1 is waiting for T2 to release its lock and similarly, transaction T2 is waiting for T1 to release its lock. All activities come to a halt state and remain at a standstill. It will remain in a standstill until the DBMS detects the deadlock and aborts one of the transactions.

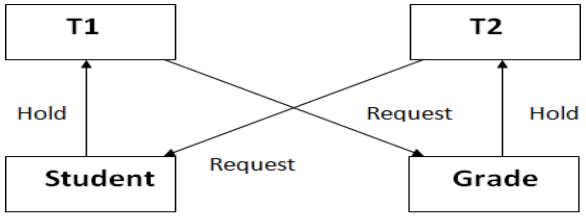


Figure: Deadlock in DBMS

Deadlock conditions (Coffman conditions)

Coffman specified four conditions for a deadlock occurrence. A deadlock may occur if all the following conditions hold true.

Mutual exclusion condition: two processes cannot use the same resource at the same time.

Hold and wait condition: A process that is holding a resource can request for additional resources that are being held by other processes in the system.

No pre-emption(book**) condition**: The process which ***once scheduled will be executed till the completion***. No other process can be scheduled by the scheduler meanwhile.

Circular wait condition: All the ***processes must be waiting for the resources in a cyclic manner*** so that the last process is waiting for the resource which is being held by the first process.

Deadlock Handling

There are three classical approaches for deadlock handling:−

- Deadlock prevention.
- Deadlock avoidance.
- Deadlock detection and removal.

a. **Deadlock Prevention**

- Deadlock prevention method is suitable for a large database. If the resources are allocated in such a way that deadlock never occurs, then the deadlock can be prevented.
- The Database management system analyzes the operations of the transaction whether they can create a deadlock situation or not. If they do, then the DBMS never allowed that transaction to be executed.
- There are two schemas for prevention

Wait-Die scheme

In this scheme, if a transaction requests for a resource which is already held with a conflicting lock by another transaction then the DBMS simply checks the timestamp of both transactions. It allows the older transaction to wait until the resource is available for execution.

Let's assume there are two transactions Ti and Tj and let TS(T) is a timestamp of any transaction T. If T2 holds a lock by some other transaction and T1 is requesting for resources held by T2 then the following actions are performed by DBMS:

1. Check if TS(Ti) < TS(Tj) - If Ti is the older transaction and Tj has held some resource, then Ti is allowed to wait until the data-item is available for execution. That means if the older transaction is waiting for a resource which is locked by the younger transaction, then the older transaction is allowed to wait for resource until it is available.
2. Check if $TS(T_i)$ < TS(Tj) - If Ti is older transaction and has held some resource and if Tj is waiting for it, then Tj is killed and restarted later with the random delay but with the same timestamp.

Wound wait scheme

- In wound wait scheme, if the older transaction requests for a resource which is held by the younger transaction, then older transaction forces younger one to kill the transaction and release the resource. After the minute delay, the younger transaction is restarted but with the same timestamp.
- If the older transaction has held a resource which is requested by the Younger transaction, then the younger transaction is asked to wait until older releases it.

b. Deadlock detection

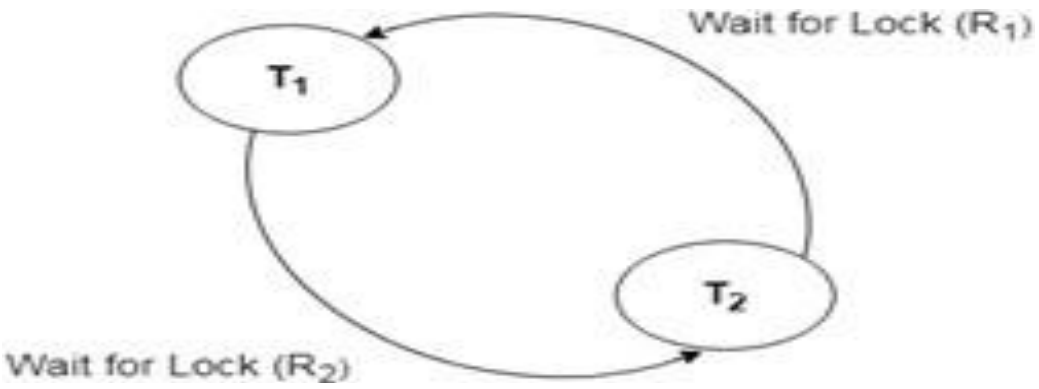
In a database, when a transaction waits indefinitely to obtain a lock, then the DBMS should detect whether the transaction is involved in a deadlock or not.

The lock manager maintains a Wait for the graph to detect the deadlock cycle in the database.

Wait for Graph

- This is the suitable method for deadlock detection. In this method, a graph is created based on the transaction and its lock. If the created graph has a cycle or closed-loop, then there is a deadlock.
- The wait for the graph is maintained by the system for every transaction which is waiting for some data held by the others. The system keeps checking the graph if there is any cycle in the graph.

The wait for a graph for the above scenario is shown below:



Here, we can use any of the two following approaches −

- First, do not allow any request for an item, which is already locked by another transaction. This is not always feasible and may cause starvation, where a transaction indefinitely waits for a data item and can never acquire it.
- The second option is to roll back one of the transactions. It is not always feasible to roll back the younger transaction, as it may be important than the older one. With the help of some relative algorithm, a transaction is chosen, which is to be aborted. This transaction is known as the **victim** and the process is known as **victim selection**.

Some of the methods used for victim selection are −

- Choose the youngest transaction.
- Choose the transaction with the fewest data items.
- Choose the transaction that has performed the least number of updates.
- Choose the transaction having the least restart overhead.
- Choose the transaction which is common to two or more cycles.

This approach is primarily suited for systems having transactions low and where fast response to lock requests is needed.

c. **Deadlock Avoidance –**

When a database is stuck in a deadlock, It is always better to avoid the deadlock rather than restarting or aborting the database. The deadlock avoidance method is suitable for smaller databases whereas the deadlock prevention method is suitable for larger databases.

One method of avoiding deadlock is using application-consistent logic. In the above-given example, Transactions that access Students and Grades should always access the tables in the same order. In this way, in the scenario described above, Transaction T1 simply waits for transaction T2 to release the lock on Grades before it begins. When transaction T2 releases the lock, Transaction T1 can proceed freely.

Another method for avoiding deadlock is to apply both row-level locking mechanism and READ COMMITTED isolation level. However, It does not guarantee to remove deadlocks completely.

9. Multiple Granularity

Granularity: It is the size of data item allowed to lock.

- Multiple Granularity can be defined as hierarchically breaking up the database into blocks which can be locked.
- The Multiple Granularity protocol enhances concurrency and reduces lock overhead.
- It maintains the track of what to lock and how to lock.
- It makes easy to decide either to lock a data item or to unlock a data item. This type of hierarchy can be graphically represented as a tree.

For example: Consider a tree which has four levels of nodes.

- The first level or higher level shows the entire database.
- The second level represents a node of type area. The higher level database consists of exactly these areas.
- The area consists of children nodes which are known as files. No file can be present in more than one area.
- Finally, each file contains child nodes known as records. The file has exactly those records that are its child nodes. No records represent in more than one file.
- Hence, the levels of the tree starting from the top level are as follows:
    1. Database
    2. Area
    3. File
    4. Record

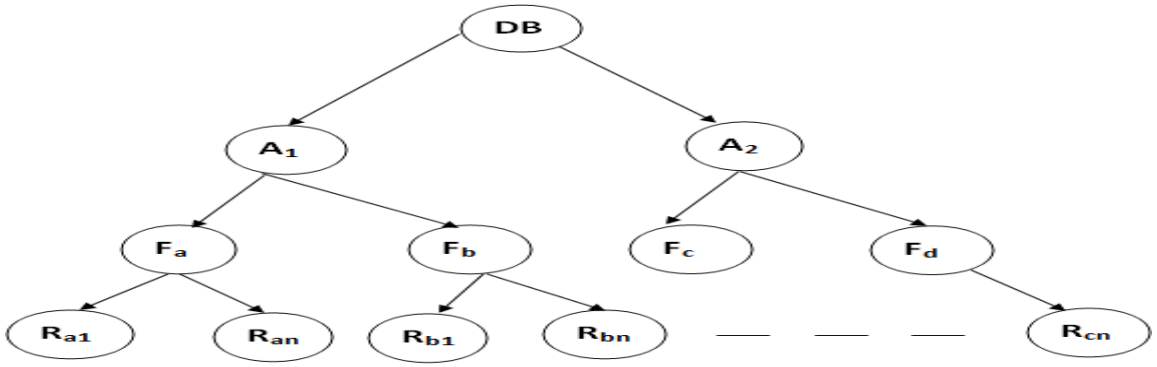


Figure: Multi Granularity tree Hierarchy

In this example, the highest level shows the entire database. The levels below are file, record, and fields.

There are three additional lock modes with multiple granularity:

Intention Mode Lock

Intention-shared (IS): It contains explicit locking at a lower level of the tree but only with shared locks.

Intention-Exclusive (IX): It contains explicit locking at a lower level with exclusive or shared locks.

Shared & Intention-Exclusive (SIX): In this lock, the node is locked in shared mode, and some node is locked in exclusive mode by the same transaction.

Compatibility Matrix with Intention Lock Modes: The below table describes the compatibility matrix for these lock modes:

	IS	IX	S	SIX	X
IS	✓	✓	✓	✓	X
IX	✓	✓	X	X	X
S	✓	X	✓	X	X
SIX	✓	X	X	X	X
X	X	X	X	X	X

It uses the intention lock modes to ensure serializability. It requires that if a transaction attempts to lock a node, then that node must follow these protocols:

- Transaction T1 should follow the lock-compatibility matrix.
- Transaction T1 firstly locks the root of the tree. It can lock it in any mode.
- If T1 currently has the parent of the node locked in either IX or IS mode, then the transaction T1 will lock a node in S or IS mode only.
- If T1 currently has the parent of the node locked in either IX or SIX modes, then the transaction T1 will lock a node in X, SIX, or IX mode only.
- If T1 has not previously unlocked any node only, then the Transaction T1 can lock a node.
- If T1 currently has none of the children of the node-locked only, then Transaction T1 will unlock a node.

Observe that in multiple-granularity, the locks are acquired in top-down order, and locks must be released in bottom-up order.

- If transaction T1 reads record R_{a9} in file F_a, then transaction T1 needs to lock the database, area A_1 and file F_a in IX mode. Finally, it needs to lock R_{a2} in S mode.
- If transaction T2 modifies record R_{a9} in file F_a, then it can do so after locking the database, area A_1 and file F_a in IX mode. Finally, it needs to lock the R_{a9} in X mode.
- If transaction T3 reads all the records in file F_a, then transaction T3 needs to lock the database, and area A in IS mode. At last, it needs to lock F_a in S mode.
- If transaction T4 reads the entire database, then T4 needs to lock the database in S mode.

10. Time-based Protocols

According to this protocol, *every transaction has a timestamp attached to it*. The timestamp is based on the time in which the transaction is entered into the system. There is read and write timestamps associated with every transaction which consists of the time at which the latest read and write operations are performed respectively.

Timestamp Ordering Protocol:

The timestamp ordering protocol uses timestamp values of the transactions to resolve the conflicting pairs of operations. Thus, ensuring serializability among transactions. Following are the denotations of the terms used to define the protocol for transaction A on the data item DT:

Terms	Denotations
Timestamp of transaction A	TS(A)
Read time-stamp of data-item DT	R-timestamp(DT)
Write time-stamp of data-item DT	W-timestamp(DT)

Following are the rules on which the Time-ordering protocol works:

1. **When transaction A is going to perform a read operation on data item DT:**

TS(A) < W-timestamp(DT): Transaction will rollback. If the timestamp of transaction A at which it has entered in the system is less than the write timestamp of DT that is the latest time at which DT has been updated then the transaction will roll back.

TS(A)=5	**TS(X)=10**
	W(DT)(already performed) so **W-timestamp(DT)=10**
Request for R(DT)	

TS(A) >= W-timestamp(DT): Transaction will be executed. If the timestamp of transaction A at which it has entered in the system is greater than or equal to the write timestamp of DT that is the latest time at which DT has been updated then the read operation will be executed.All data-item timestamps updated.

R-timestamp(DT) will be max(R-timestamp(DT), TS(A))

TS(A)=10	**TS(X)=5**
	W(DT)(already performed) so **W-timestamp(DT)=5**
Request for R(DT)	

2. **When transaction A is going to perform a write operation on data item DT:**

TS(A) < R-timestamp(DT): Transaction will rollback. If the timestamp of transaction A at which it has entered in the system is less than the read timestamp of DT that is the latest time at which DT has been read then the transaction will rollback.

TS(A)=5	**TS(X)=10**
	R(DT)(already performed) so **R-timestamp(DT)=10**
Request for W(DT)	

TS(A) < W-timestamp(DT): Transaction will rollback. If the timestamp of transaction A at which it has entered in the system is less than the write timestamp of DT that is the latest time at which DT has been updated then the transaction will rollback else W-timestamp(DT) will be max(W-timestamp(DT), TS(A))

TS(A)=5	**TS(X)=10**
	W(DT)(already performed) so **W-timestamp(DT)=10**
Request for W(DT)	

All the operations other than this will be executed.
Those with timestamping will be conflict serializable and view serilizable

Thomas' Write Rule: The rule alters the timestamp-ordering protocol to *make the schedule view serializable*. For the case TS(A) < W-timestamp(DT), in the timestamp-ordering protocol, the transaction will get rollback but according to **Thomas Write Rule**, whenever the write operation comes up, it will get ignored.

Advantages and Disadvantages of TO protocol:

- TO protocol ensures serializability since the precedence graph is as follows:

Image: Precedence Graph for TS ordering

TS protocol ensures freedom from deadlock that means no transaction ever waits.

- But the schedule may not be recoverable and may not even be cascade- free.

11. Recovery System

Failure classification

To find that where the problem has occurred, we generalize a failure into the following categories:

1. Transaction failure
2. System crash
3. Disk failure

1. Transaction failure

The transaction failure occurs when it fails to execute or when it reaches a point from where it can't go any further. If a few transaction or process is hurt, then this is called as transaction failure.

Reasons for a transaction failure could be -

1. **Logical errors:** If a transaction cannot complete due to some code error or an internal error condition, then the logical error occurs.
2. **Syntax error:** It occurs where the DBMS itself terminates an active transaction because the database system is not able to execute it. **For example,** The system aborts an active transaction, in case of deadlock or resource unavailability.

2. System Crash

- System failure can occur due to power failure or other hardware or software failure. **Example:** Operating system error.

Fail-stop assumption: In the system crash, non-volatile storage is assumed not to be corrupted.

3. Disk Failure

- It occurs where hard-disk drives or storage drives used to fail frequently. It was a common problem in the early days of technology evolution.
- Disk failure occurs due to the formation of bad sectors, disk head crash, and unreachability to the disk or any other failure, which destroy all or part of disk storage.

What is recovery?

- Recovery is the process of restoring a database to the correct state in the event of a failure.
- It ensures that the database is reliable and remains in consistent state in case of a failure.

Database recovery can be classified into two parts;

1. Rolling Forward applies redo records to the corresponding data blocks.

2. Rolling Back applies rollback segments to the datafiles. It is stored in transaction tables.

- We can recover the database using Log–Based Recovery.

Log-Based Recovery

- Logs are the sequence of records, that maintain the records of actions performed by a transaction.
- In Log – Based Recovery, log of each transaction is maintained in some stable storage. If any failure occurs, it can be recovered from there to recover the database.
- The log contains the information about the transaction being executed, values that have been modified and transaction state.
- All these information will be stored in the order of execution.
  - **Example:**
    Assume, a transaction to modify the address of an employee. The following logs are written for this transaction,
    Log 1: Transaction is initiated, writes 'START' log.
    Log: <T_n START>
    Log 2: Transaction modifies the address from 'Pune' to 'Mumbai'.
    Log: <T_n Address, 'Pune', 'Mumbai'>
    Log 3: Transaction is completed. The log indicates the end of the transaction.
    Log: <T_n COMMIT>
    There are two methods of creating the log files and updating the database,
    1. Deferred Update
    2. Immediate Update
    Deferred update – This technique does not physically update the database on disk until a transaction has reached its commit point. Before reaching commit, all transaction updates are recorded in the local transaction workspace. If a transaction fails before reaching its commit point, it will not have changed the database in any way so UNDO is not needed. It may be necessary to REDO the effect of the operations that are recorded in the local transaction workspace, because their effect may not yet have been written in the database. Hence, a deferred update is also known as the **No-undo/redo algorithm**
    Immediate update – In the immediate update, the database may be updated by some operations of a transaction before the transaction reaches its commit point. However, these operations are recorded in a log on disk before they are applied to the database, making recovery still possible. If a transaction fails to reach its commit point, the effect of its operation must be undone i.e. the transaction must be rolled back hence we require both undo and redo. This technique is known as **undo/redo algorithm.**

Recovery with Concurrent Transaction

- When two transactions are executed in parallel, the logs are interleaved. It would become difficult for the recovery system to return all logs to a previous point and then start recovering.
- To overcome this situation 'Checkpoint' is used.

Checkpoint

- Checkpoint acts like a benchmark.
- Checkpoints are also called as **Syncpoints or Savepoints.**
- It is a mechanism where all the previous logs are removed from the system and stored permanently in a storage system.

- It declares a point before which the database management system was in consistent state and all the transactions were committed.
- It is a point of synchronization between the database and the transaction log file.
- It involves operations like writing log records in main memory to secondary storage, writing the modified blocks in the database buffers to secondary storage and writing a checkpoint record to the log file.
- The checkpoint record contains the identifiers of all transactions that are active at the time of the checkpoint.

Recovery

- When concurrent transactions crash and recover, the checkpoint is added to the transaction and recovery system recovers the database from failure in following manner,

  1. Recovery system reads the log files from end to start checkpoint. It can reverse the transaction.
  2. It maintains undo log and redo log.
  3. It puts the transaction in the redo log if the recovery system sees a log $<T_n$, Commit>.
  4. It puts the transaction in undo log if the recovery system sees a log with $<T_n$, Start>.
- All the transactions in the undo log are undone and their logs are removed.
- All the transactions in the redo log and their previous logs are removed and then redone before saving their logs.

Back up techniques:

Some of the backup techniques are as follows :

- **Full database backup –** In this full database including data and database, Meta information needed to restore the whole database, including full-text catalogs are backed up in a predefined time series.
- **Differential backup –** It stores only the data changes that have occurred since the last full database backup. When some data has changed many times since last full database backup, a differential backup stores the most recent version of the changed data. For this first, we need to restore a full database backup.
- **Transaction log backup –** In this, all events that have occurred in the database, like a record of every single statement executed is backed up. It is the backup of transaction log entries and contains all transactions that had happened to the database. Through this, the database can be recovered to a specific point in time. It is even possible to perform a backup from a transaction log if the data files are destroyed and not even a single committed transaction is lost.